

1. RESOLUCIÓN DE PROBLEMAS MEDIANTE BÚSQUEDA (PARTE 2)

IA 3.2 - Programación III

1º C - 2026

Dr. Mauro Lucci



Repaso

— — —

- Muchos **problemas de optimización y factibilidad** se pueden formular como **problemas de búsqueda**.
- Un **algoritmo de búsqueda** recibe un problema de búsqueda cualquiera y trata de retornar una **solución**, idealmente **óptima**.
- Problemas reales tienen **espacios de estados** grandes y su **grafo de estados** no se puede almacenar completamente en memoria.

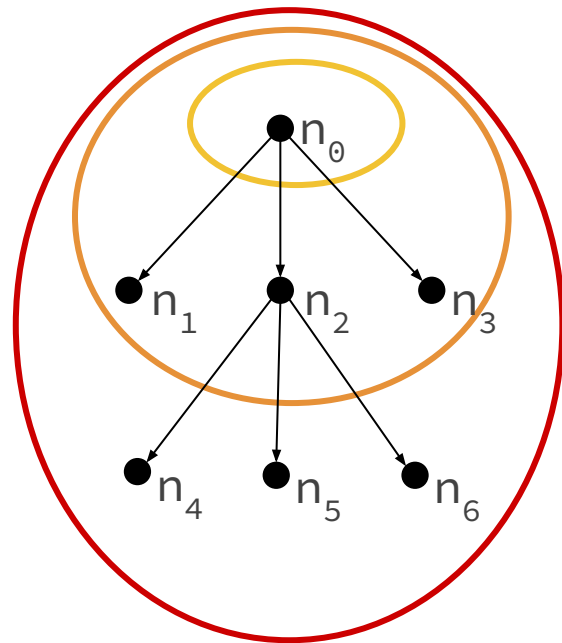


**En vez de construir el grafo de estados antes de buscar,
se busca a medida que se construye**

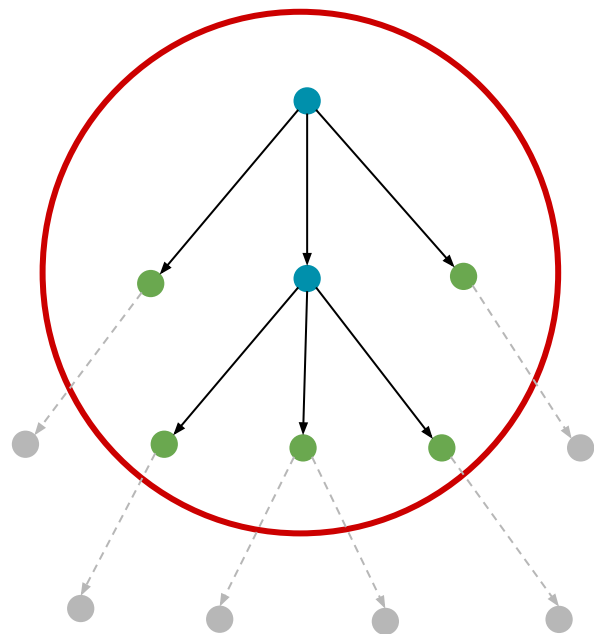


Construir Buscar

1. Buscar en n_0 .
2. Si no hay solución, **expandir** n_0 , es decir, **generar** sus vecinos.
3. **Elegir** un vecino, digamos n_2 .
4. Buscar en n_2 .
5. Si no hay solución, **expandir** n_2 .
6. ...



Clasificación de nodos



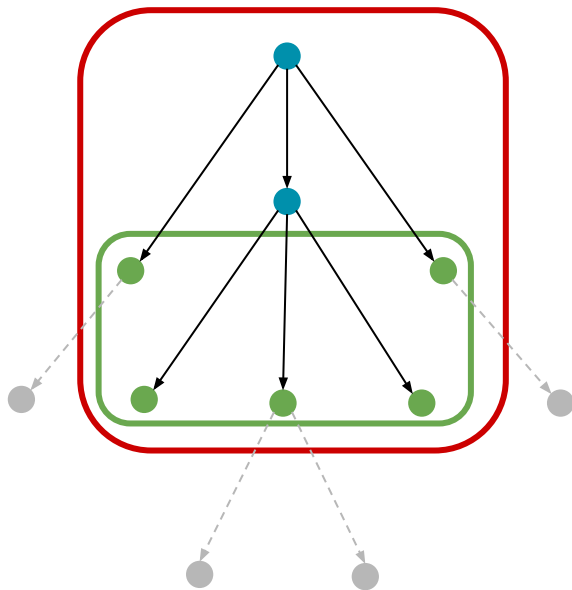
- Nodos **generados y expandidos**.
Se buscó y no se halló solución.
- Nodos **generados y no expandidos**.
Próximos nodos donde buscar.
- Nodos **no generados**.
Todavía no ocupan memoria.

Alcanzados y Frontera

FRONTERA

Conjunto con los **nodos** generados y no expandidos.

Candidatos a expandir.



ALCANZADOS

Conjunto con los **estados** de los nodos generados (expandidos o no).

Historial de la búsqueda.

Pronto veremos la diferencia entre **nodo** y **estado**. Por lo pronto, un nodo tiene un estado y cierta información extra.

Búsqueda de grafo y de árbol

— — —

Hay dos algoritmos de búsqueda clásicos:

1. **Búsqueda de grafo.**
2. **Búsqueda de árbol.**

Búsqueda de grafo

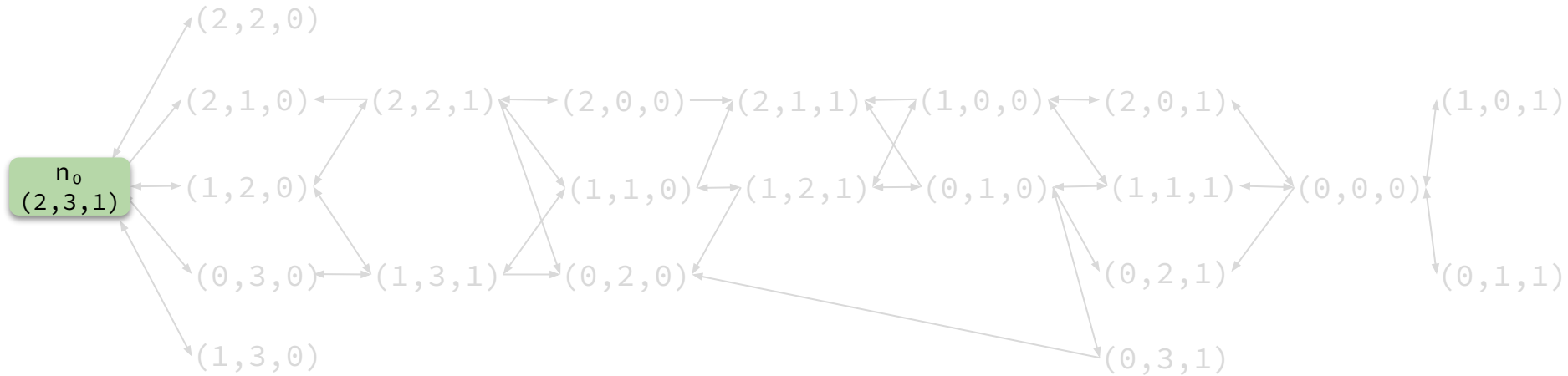
Búsqueda de grafo

1. **Inicialización.** Generar la raíz n_0 con el estado inicial s_0 . Poner en ALCANZADOS a s_0 y en FRONTERA a n_0 .
2. **Iteración.** Mientras FRONTERA no esté vacía:
3. **Selección.** Sacar un nodo n de FRONTERA.
4. **Test objetivo.** Retornar si n tiene un estado objetivo.
5. **Expansión.** Expandir n y actualizar FRONTERA Y ALCANZADOS con sus vecinos no alcanzados.



Ejemplo – Cruce de río

— — —



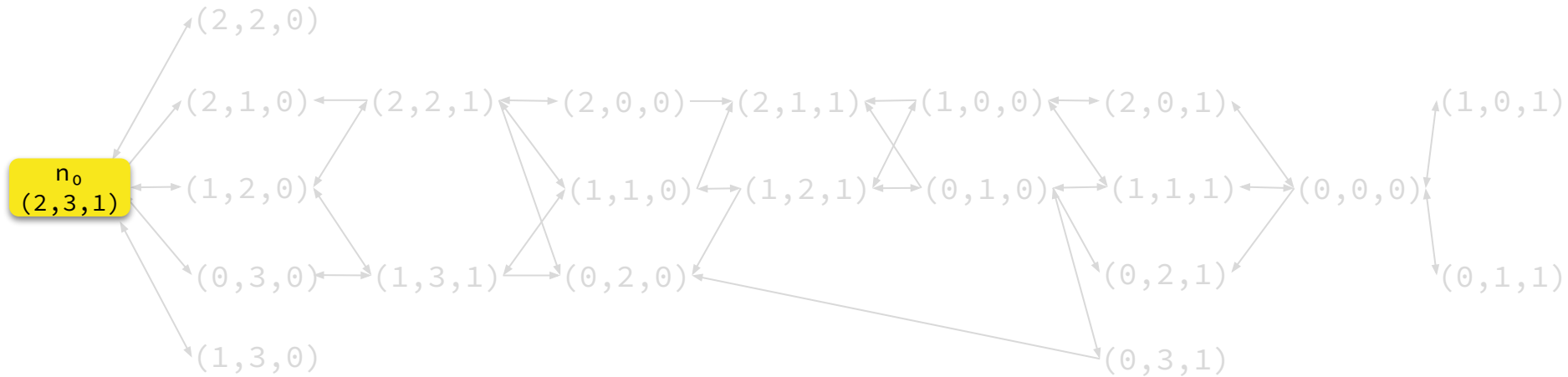
FRONTERA = $\{n_0\}$

ALCANZADOS = $\{(2,3,1)\}$



Ejemplo – Cruce de río

— — —

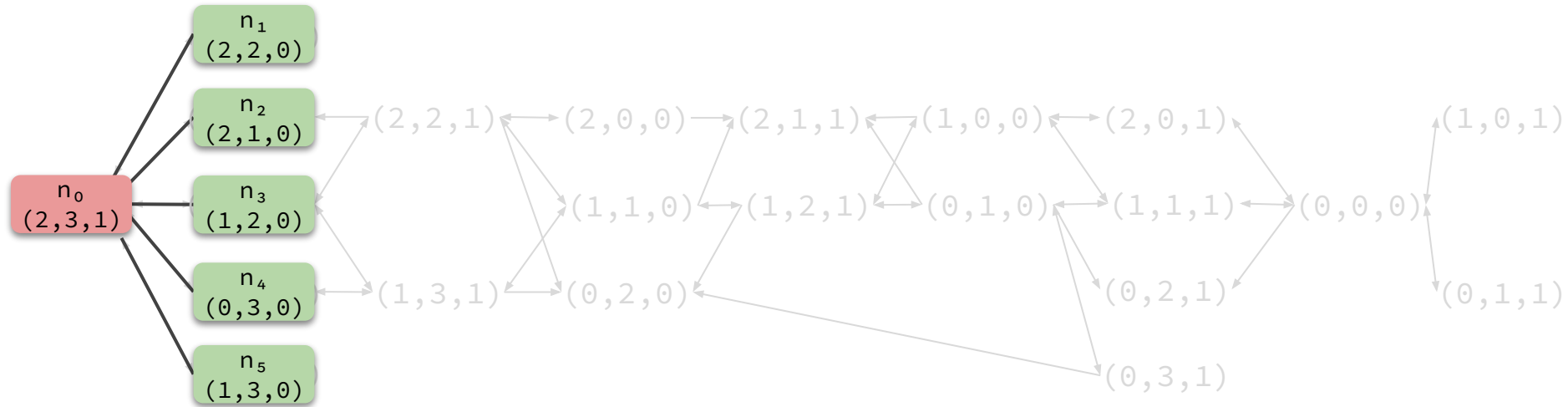


FRONTERA = $\{\}$

ALCANZADOS = $\{(2,3,1)\}$



Ejemplo – Cruce de río

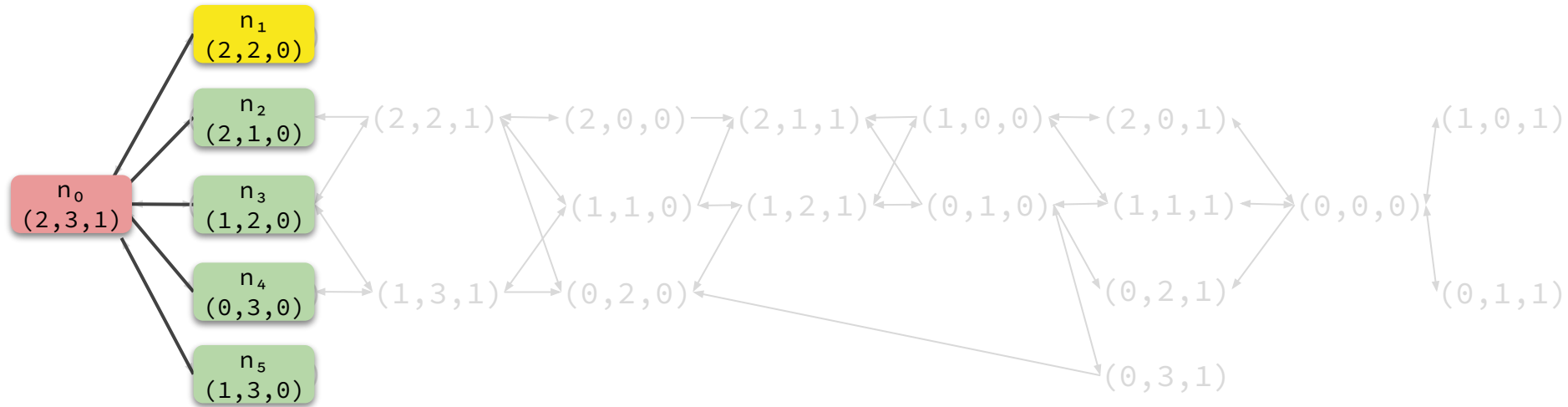


FRONTERA = $\{n_1, n_2, n_3, n_4, n_5\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0)\}$



Ejemplo – Cruce de río

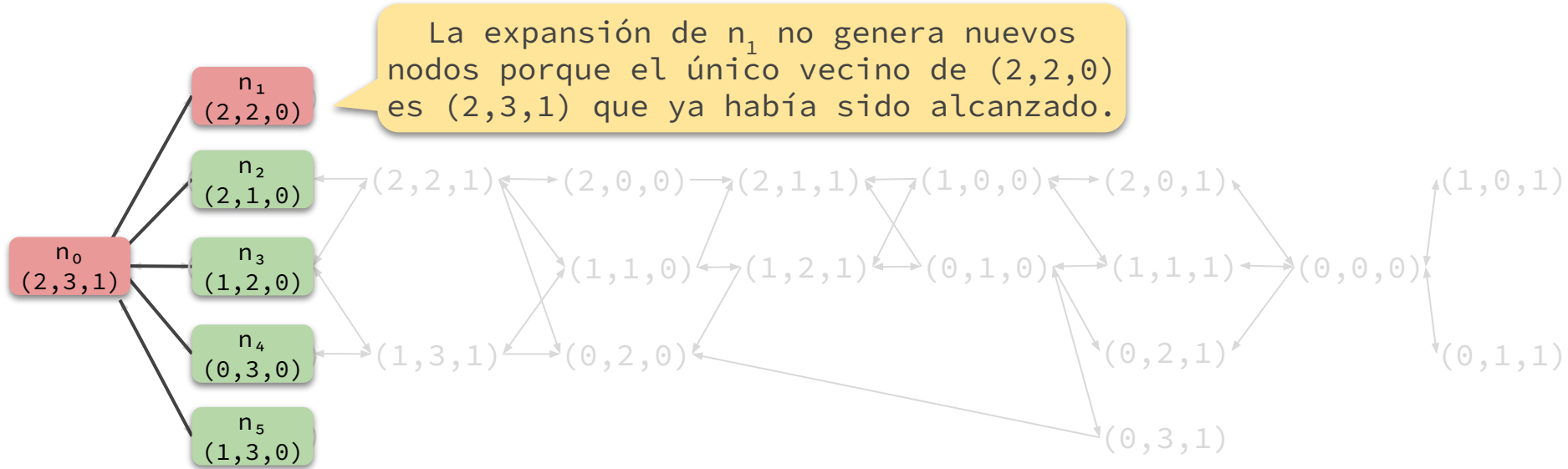


FRONTERA = $\{n_2, n_3, n_4, n_5\}$

ALCANZADOS = $\{(2, 3, 1), (2, 2, 0), (2, 1, 0), (1, 2, 0), (0, 3, 0), (1, 3, 0)\}$

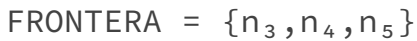


Ejemplo – Cruce de río



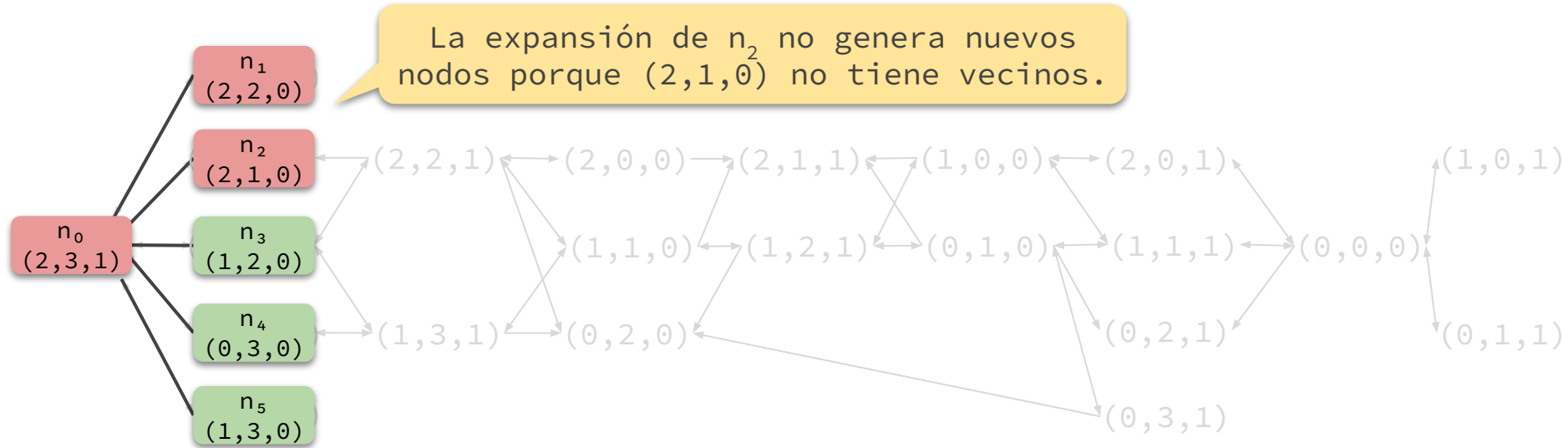
FRONTERA = $\{n_2, n_3, n_4, n_5\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0)\}$


$$\text{ALCANZADOS} = \{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0)\}$$



Ejemplo – Cruce de río

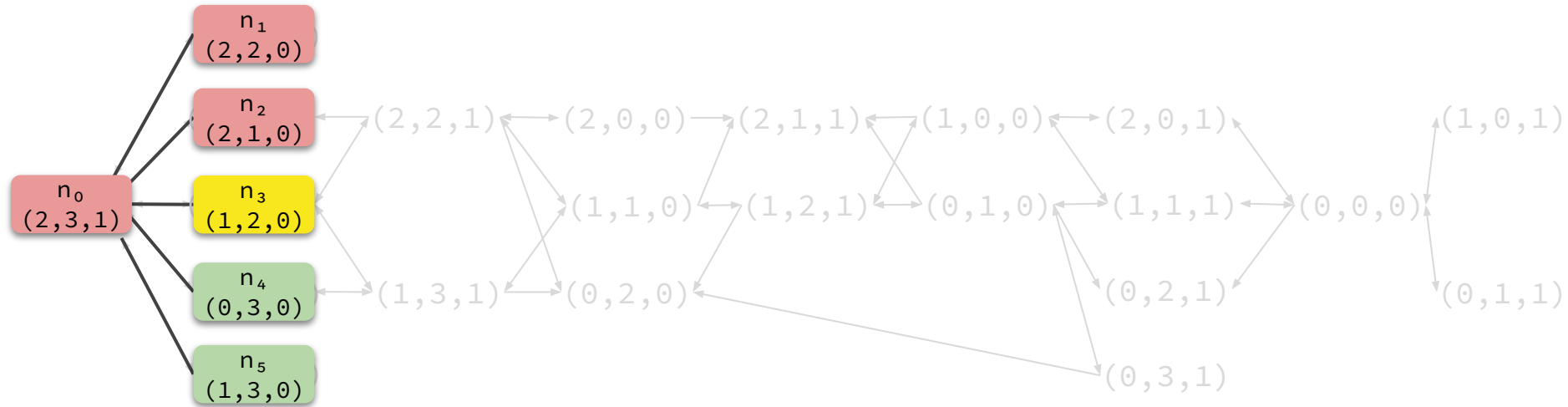


FRONTERA = $\{n_3, n_4, n_5\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0)\}$



Ejemplo – Cruce de río

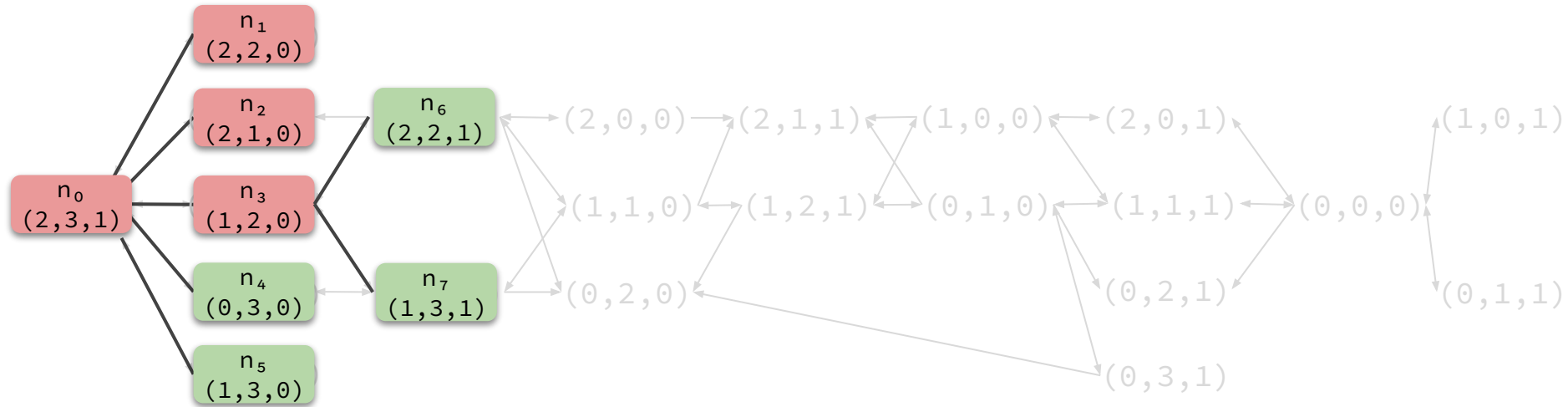


FRONTERA = $\{n_4, n_5\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0)\}$



Ejemplo – Cruce de río

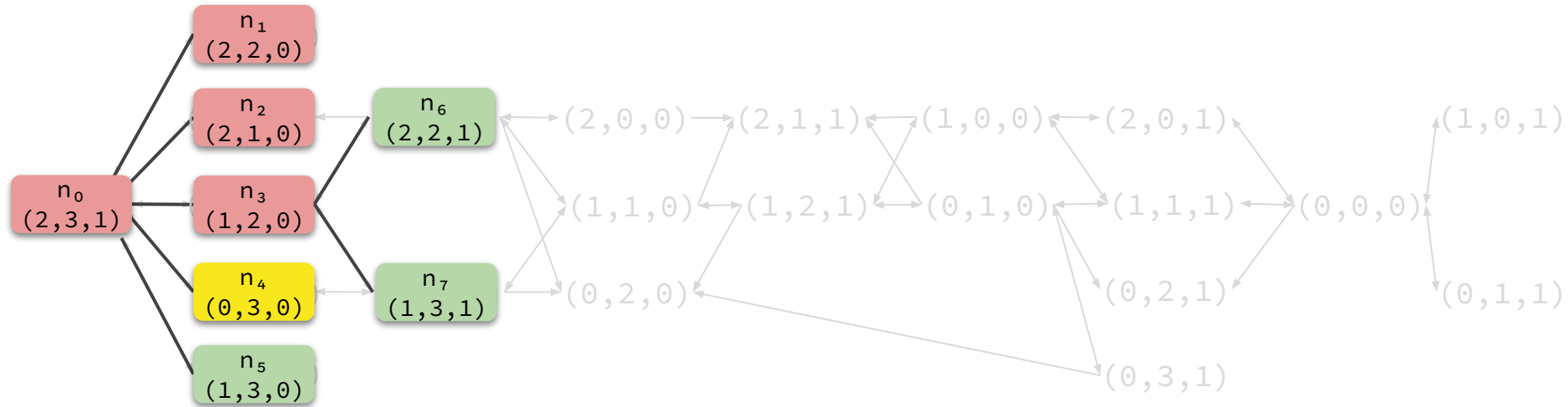


FRONTERA = $\{n_4, n_5, n_6, n_7\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1)\}$



Ejemplo – Cruce de río

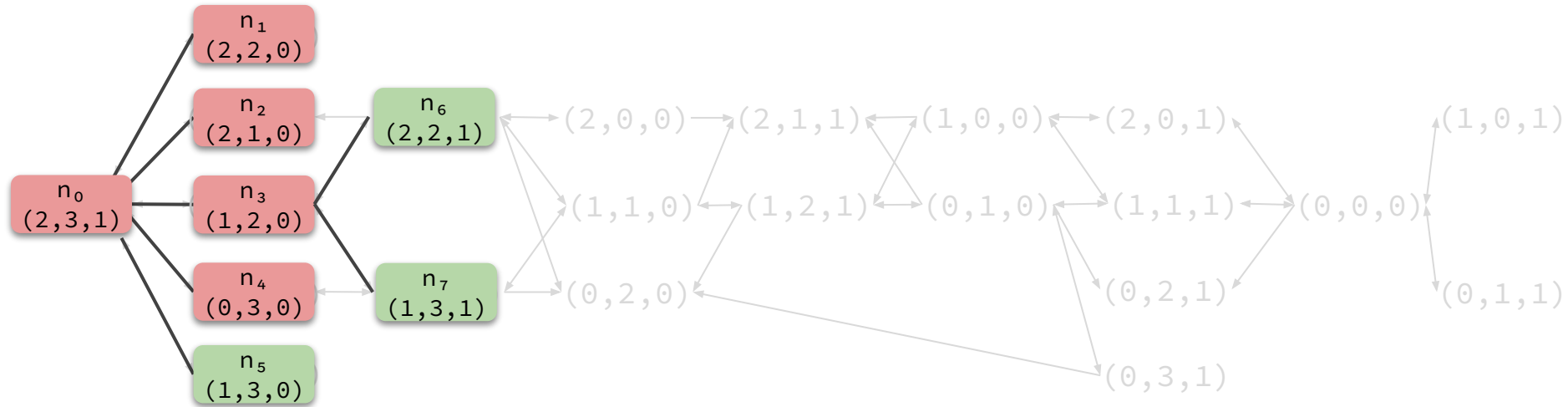


FRONTERA = $\{n_5, n_6, n_7\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1)\}$



Ejemplo – Cruce de río

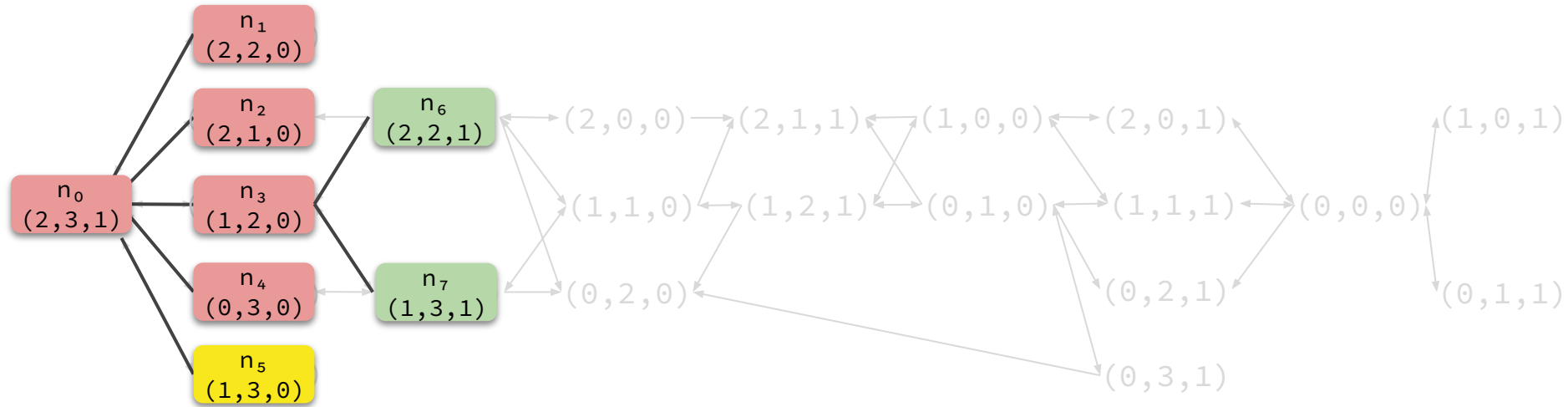


FRONTERA = $\{n_5, n_6, n_7\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1)\}$



Ejemplo – Cruce de río

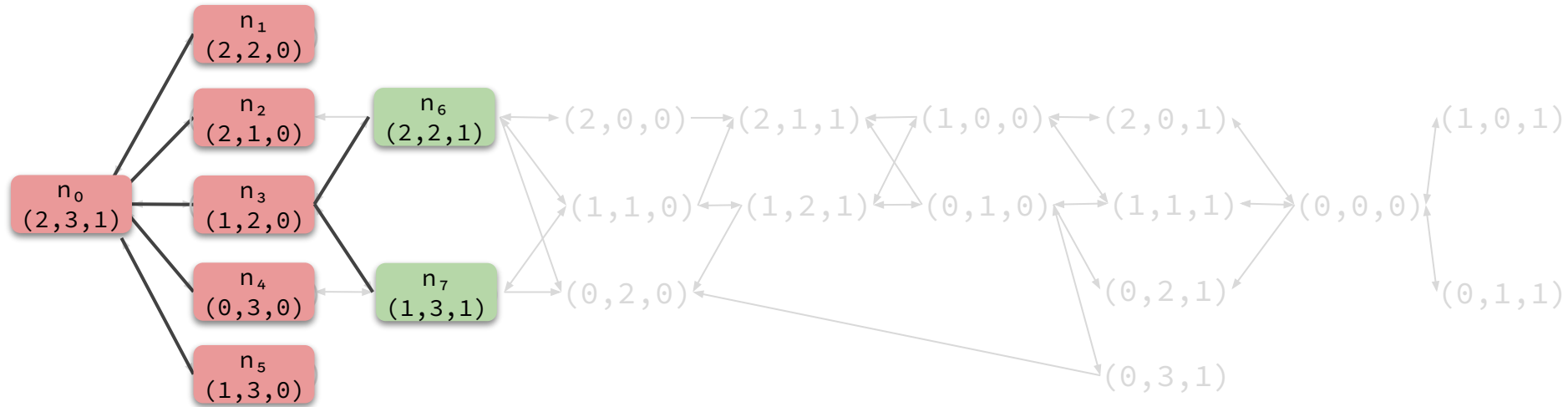


FRONTERA = $\{n_6, n_7\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1)\}$



Ejemplo – Cruce de río

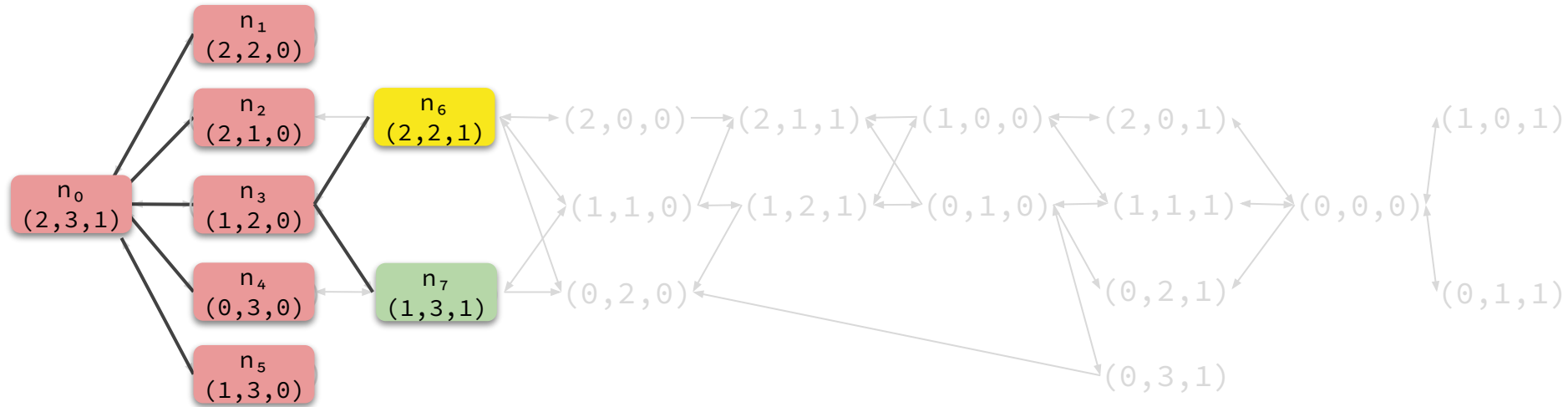


FRONTERA = $\{n_6, n_7\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1)\}$



Ejemplo – Cruce de río

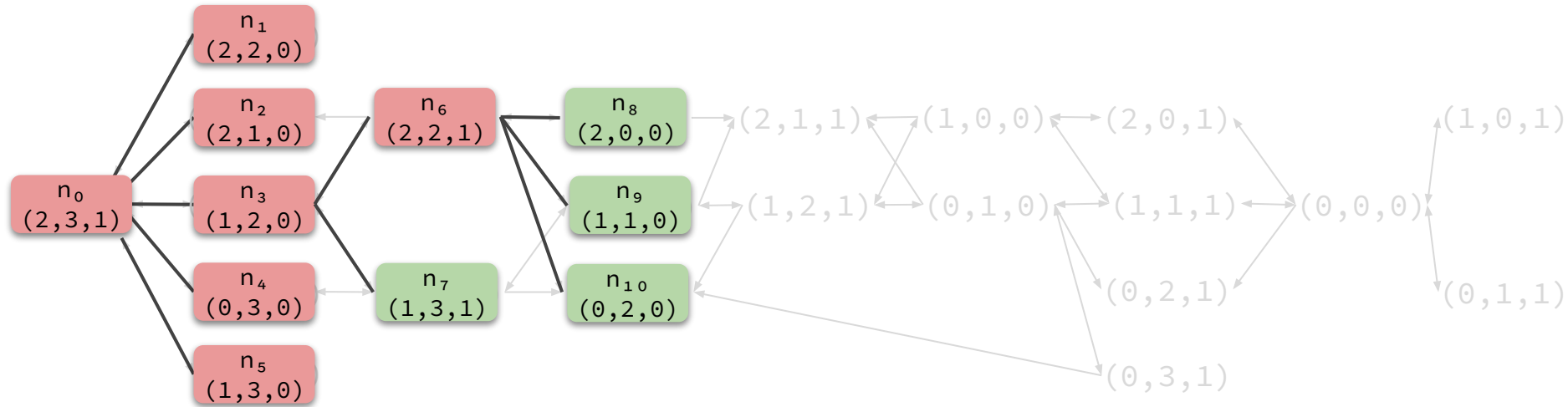


FRONTERA = $\{n_7\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1)\}$



Ejemplo – Cruce de río

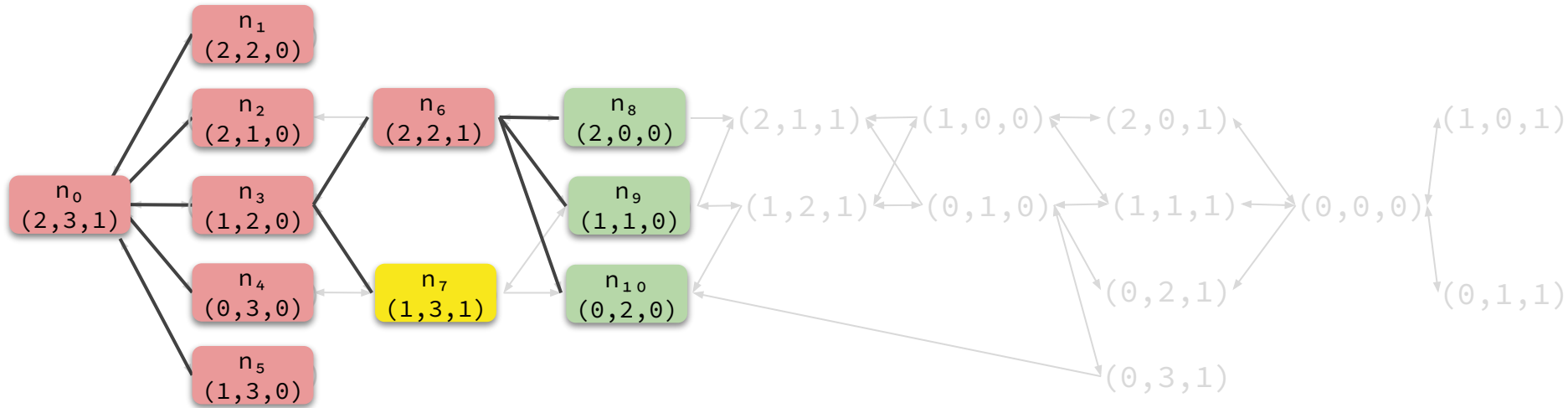


FRONTERA = $\{n_7, n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0)\}$



Ejemplo – Cruce de río

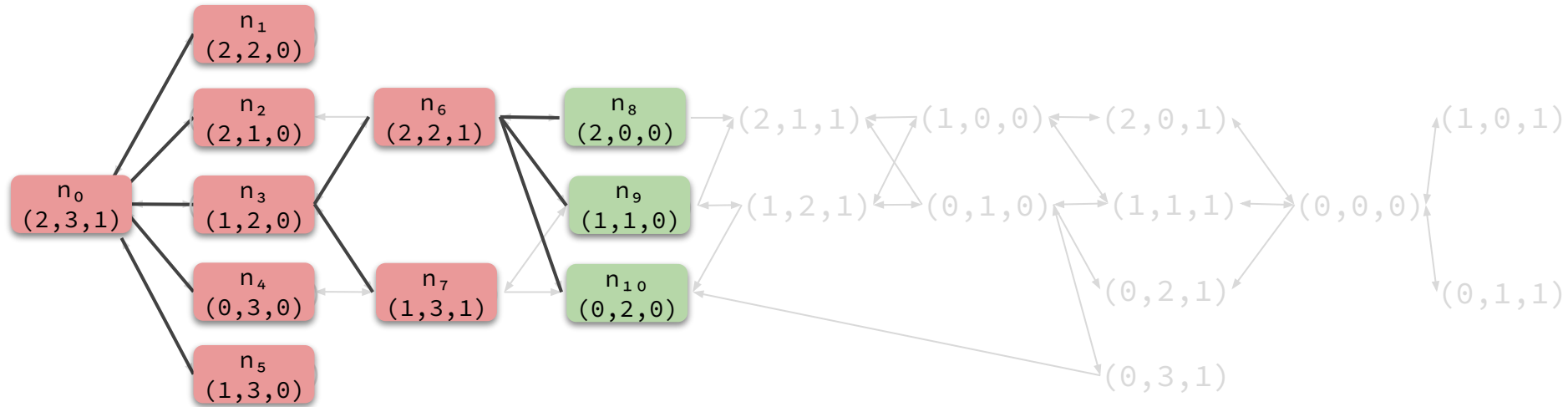


FRONTERA = $\{n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0)\}$



Ejemplo – Cruce de río

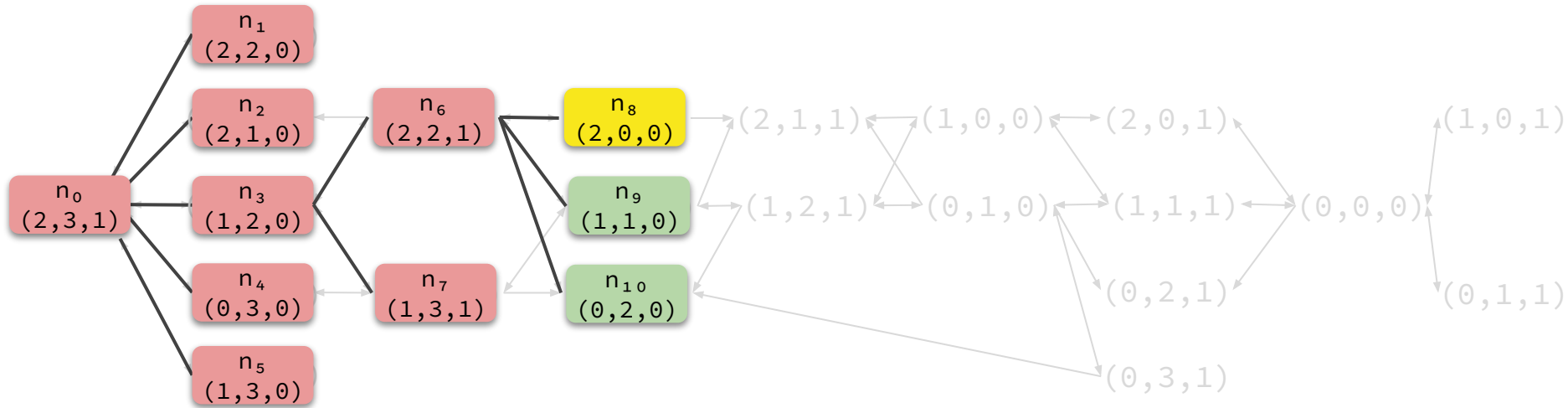


FRONTERA = $\{n_8, n_9, n_{10}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0)\}$



Ejemplo – Cruce de río

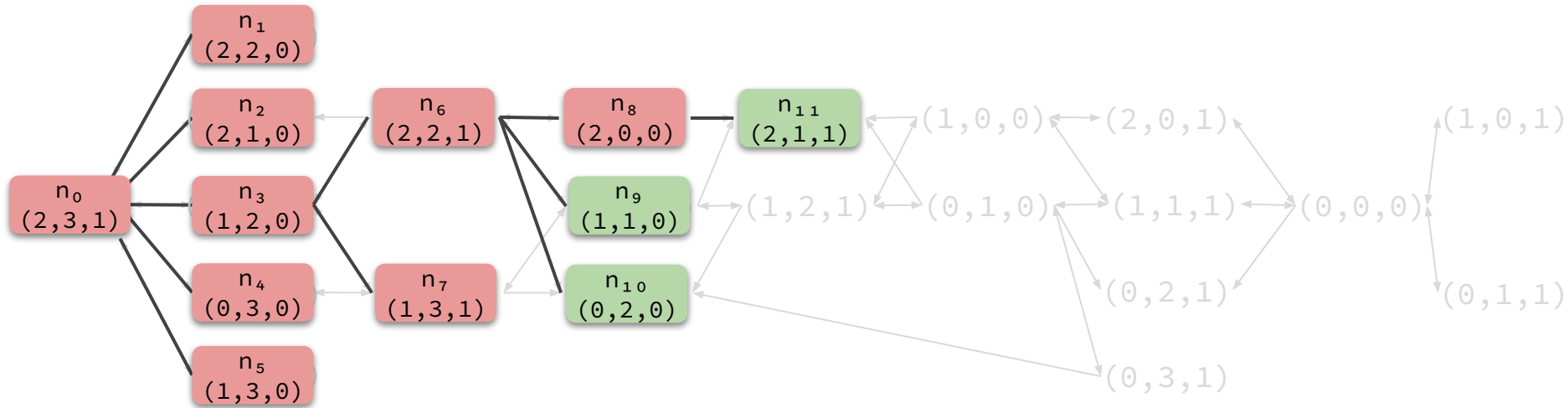


FRONTERA = $\{n_9, n_{10}\}$

ALCANZADOS = $\{(2, 3, 1), (2, 2, 0), (2, 1, 0), (1, 2, 0), (0, 3, 0), (1, 3, 0), (2, 2, 1), (1, 3, 1), (2, 0, 0), (1, 1, 0), (0, 2, 0)\}$



Ejemplo – Cruce de río

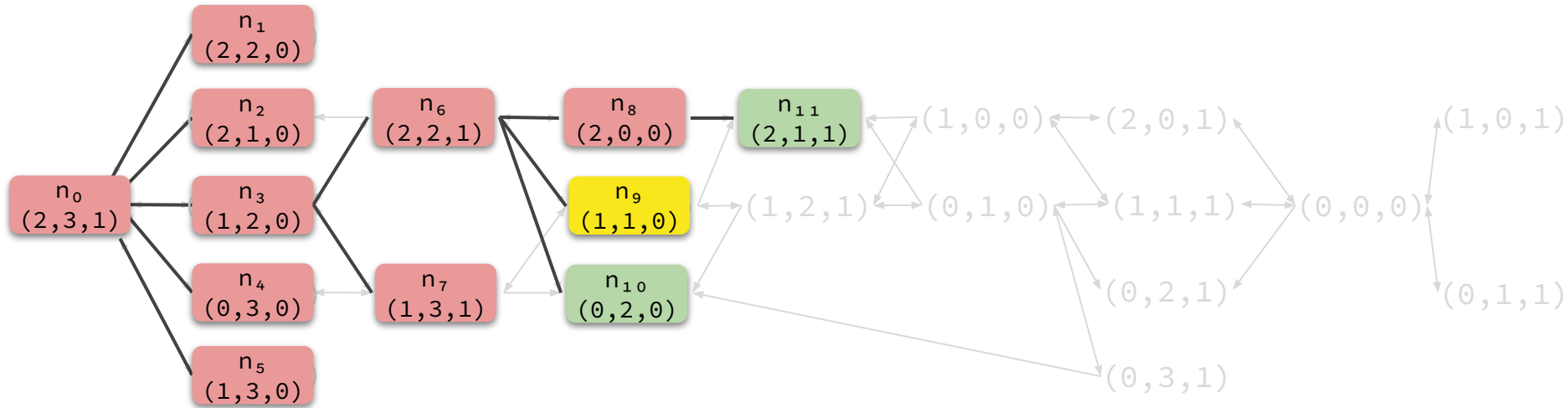


FRONTERA = $\{n_9, n_{10}, n_{11}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1)\}$



Ejemplo – Cruce de río

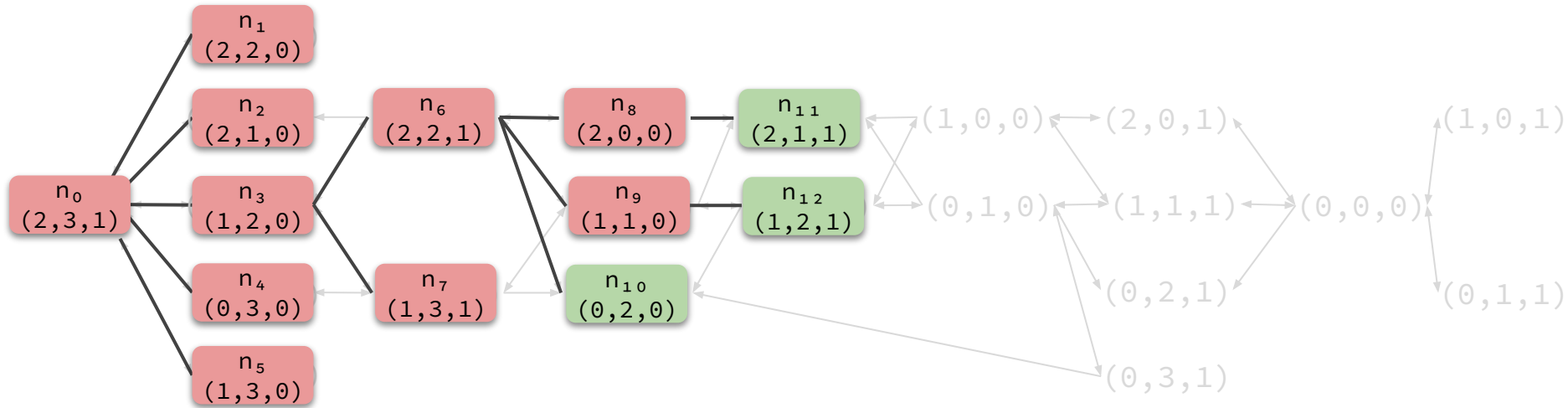


FRONTERA = $\{n_{10}, n_{11}\}$

ALCANZADOS = $\{(2, 3, 1), (2, 2, 0), (2, 1, 0), (1, 2, 0), (0, 3, 0), (1, 3, 0), (2, 2, 1), (1, 3, 1), (2, 0, 0), (1, 1, 0), (0, 2, 0), (2, 1, 1)\}$



Ejemplo – Cruce de río

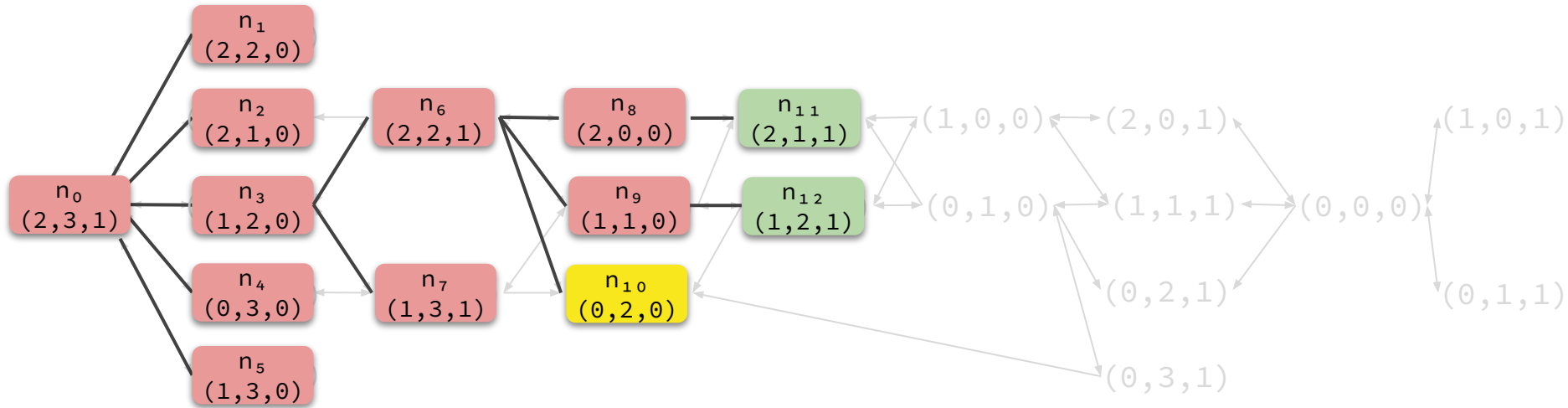


FRONTERA = $\{n_{10}, n_{11}, n_{12}\}$

ALCANZADOS = $\{(2, 3, 1), (2, 2, 0), (2, 1, 0), (1, 2, 0), (0, 3, 0), (1, 3, 0), (2, 2, 1), (1, 3, 1), (2, 0, 0), (1, 1, 0), (0, 2, 0), (2, 1, 1), (1, 2, 1)\}$



Ejemplo – Cruce de río

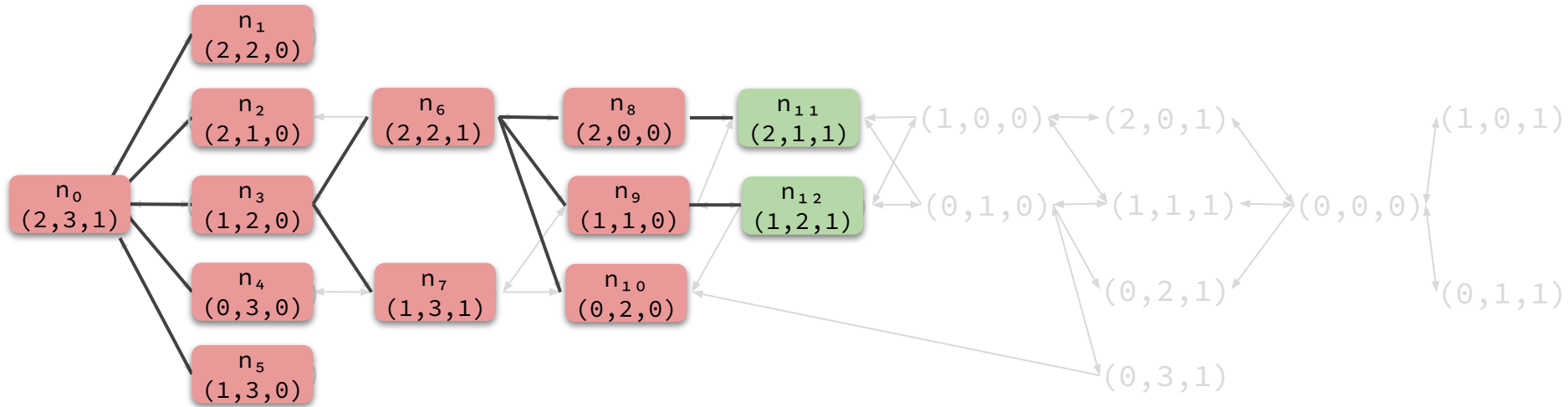


FRONTERA = $\{n_{11}, n_{12}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1)\}$



Ejemplo – Cruce de río

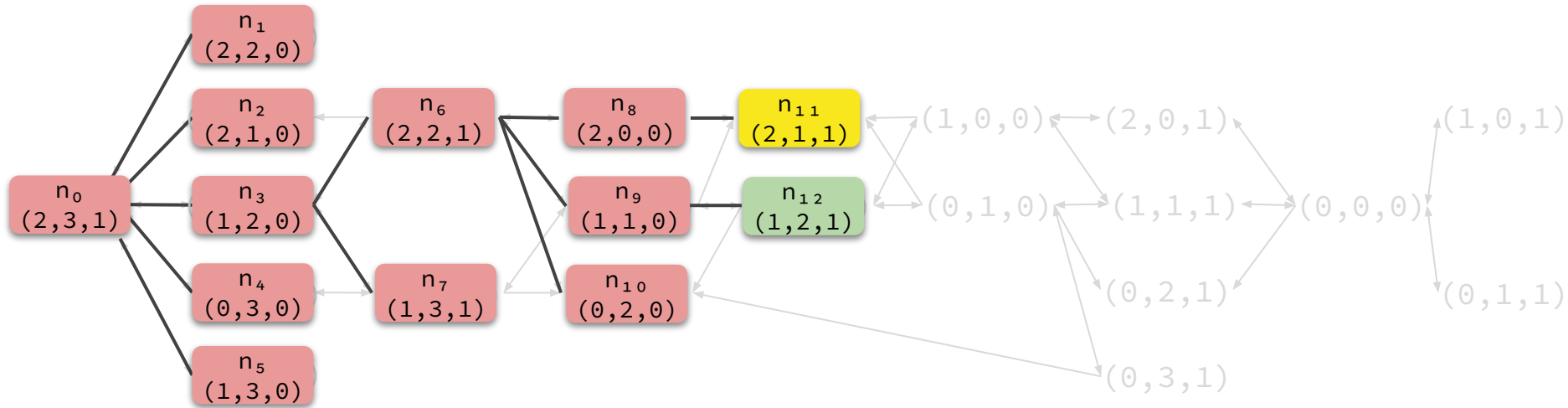


FRONTERA = $\{n_{11}, n_{12}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1)\}$



Ejemplo – Cruce de río

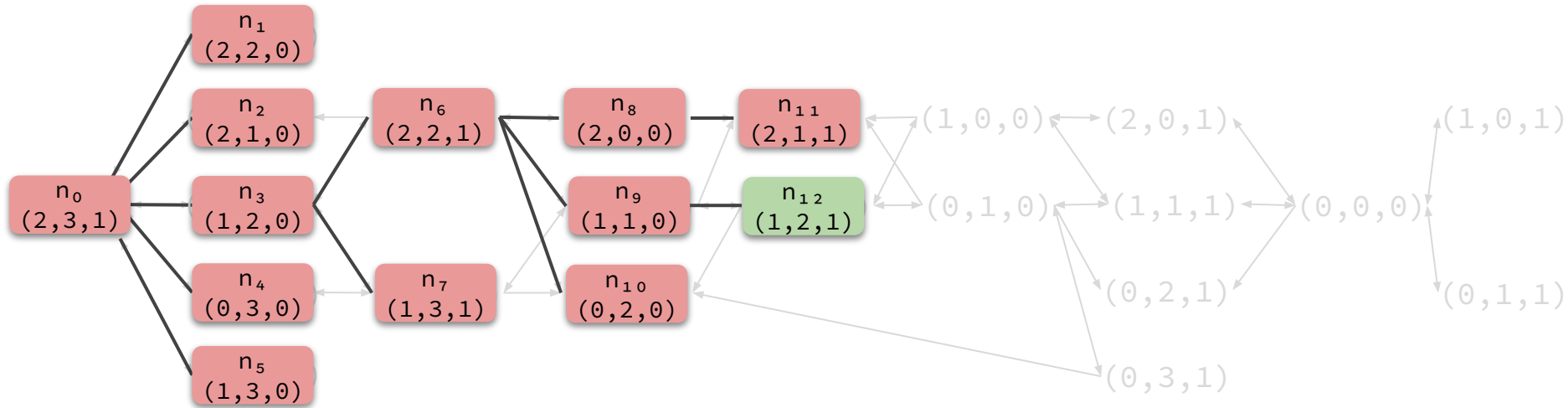


FRONTERA = $\{n_{12}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1)\}$



Ejemplo – Cruce de río

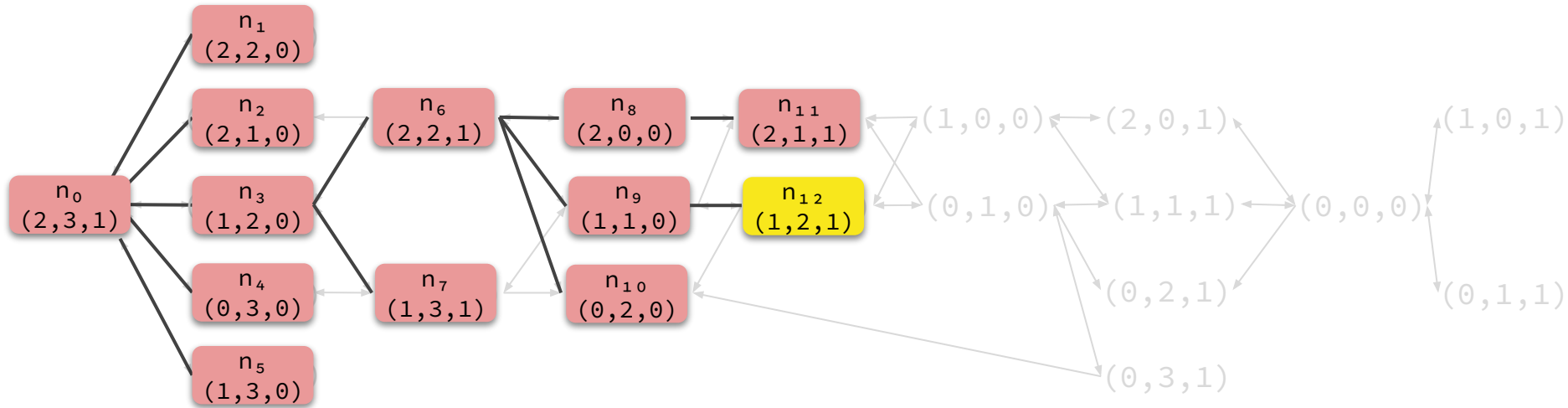


FRONTERA = $\{n_{12}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1)\}$



Ejemplo – Cruce de río

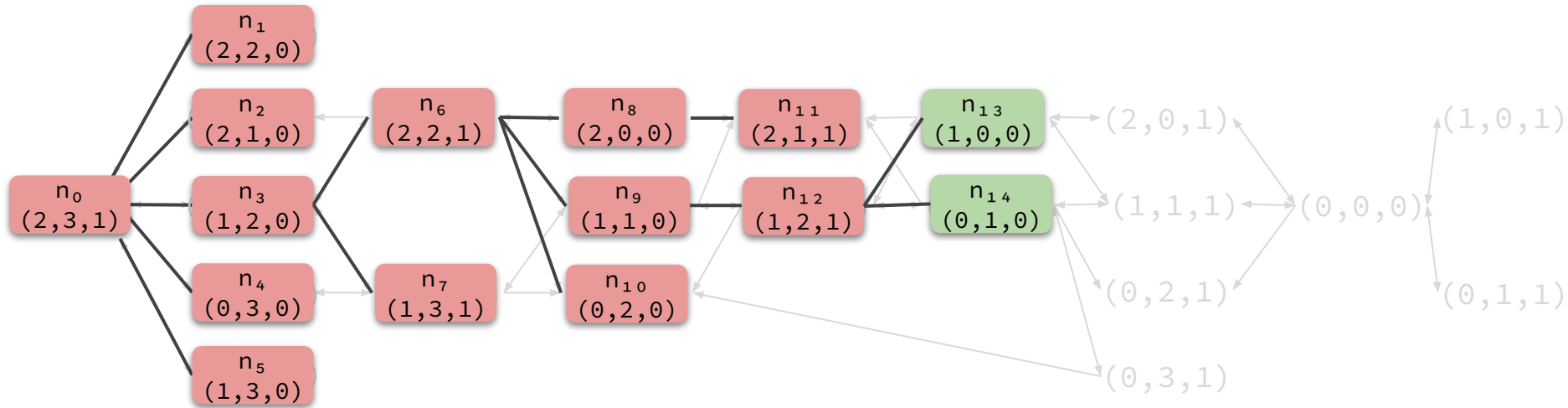


FRONTERA = {}

ALCANZADOS = {(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1)}



Ejemplo – Cruce de río

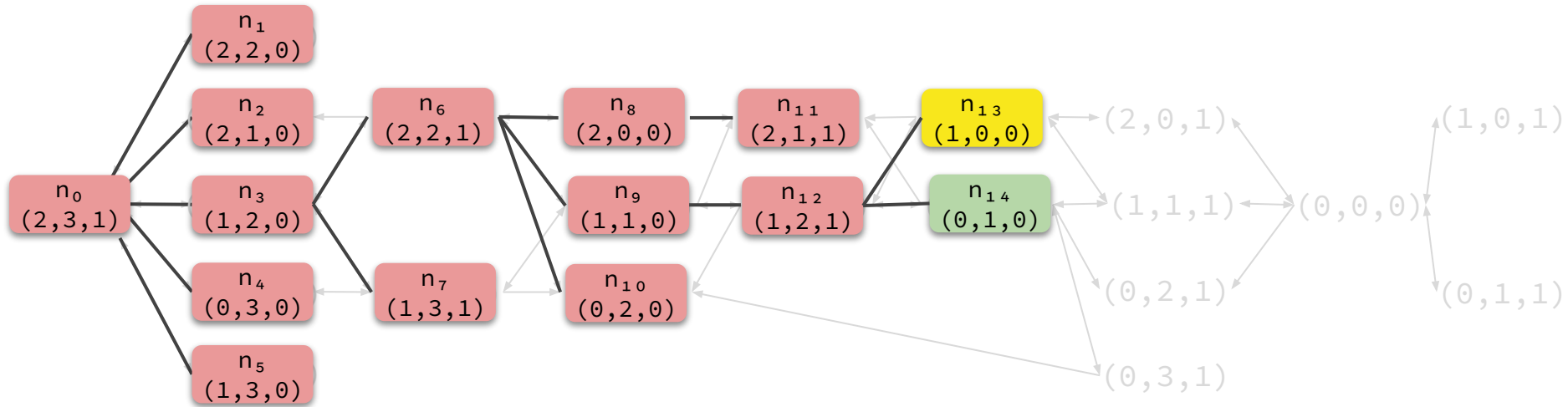


FRONTERA = $\{n_{13}, n_{14}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0)\}$



Ejemplo – Cruce de río

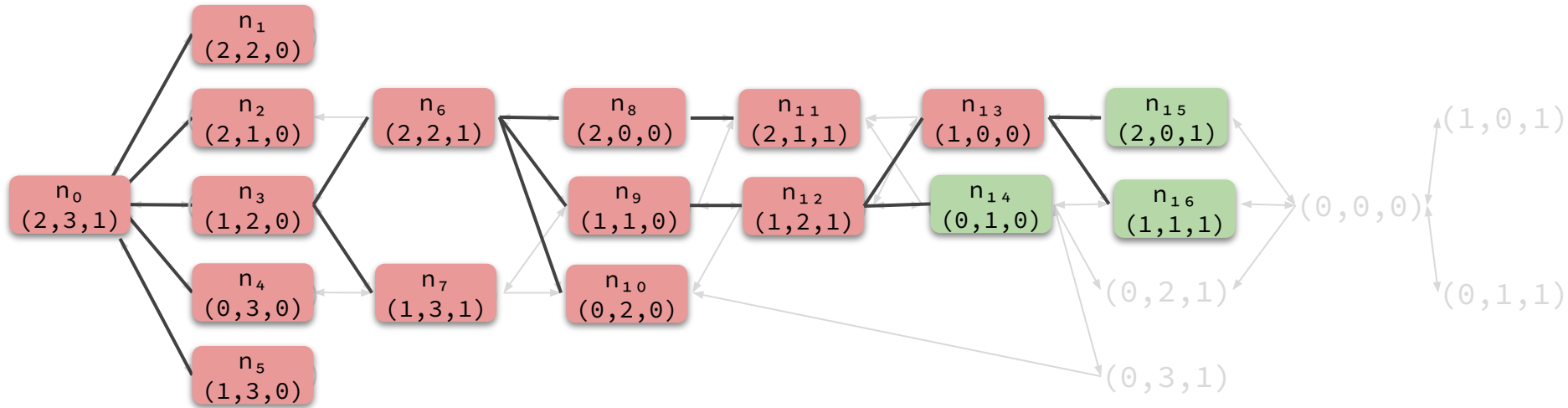


FRONTERA = $\{n_{14}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0)\}$



Ejemplo – Cruce de río

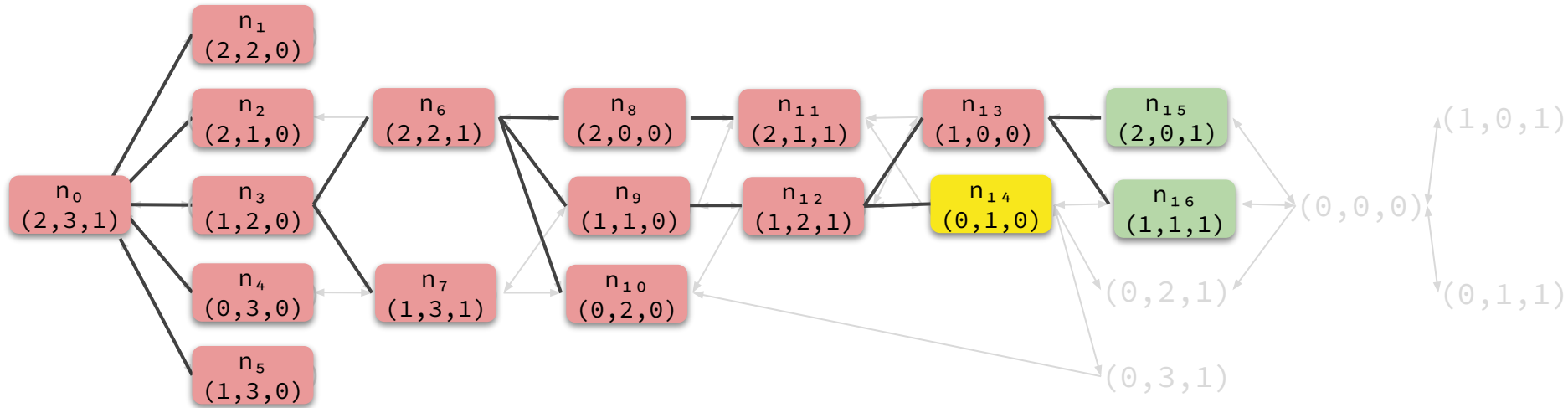


FRONTERA = $\{n_{14}, n_{15}, n_{16}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (0,1,0)\}$



Ejemplo – Cruce de río

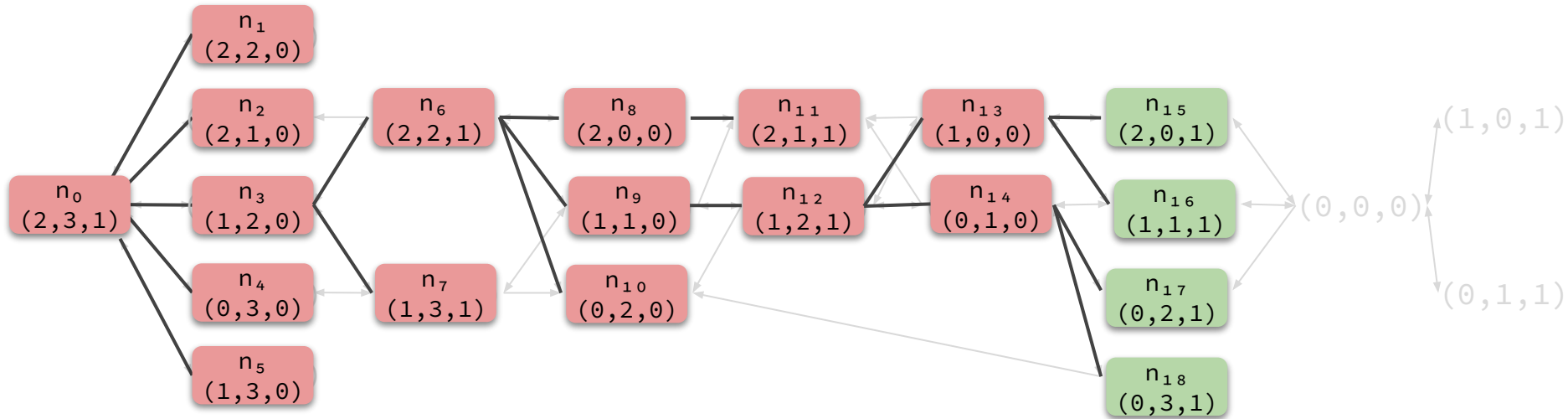


FRONTERA = $\{n_{15}, n_{16}\}$

ALCANZADOS = $\{(2, 3, 1), (2, 2, 0), (2, 1, 0), (1, 2, 0), (0, 3, 0), (1, 3, 0), (2, 2, 1), (1, 3, 1), (2, 0, 0), (1, 1, 0), (0, 2, 0), (2, 1, 1), (1, 2, 1), (1, 0, 0), (0, 1, 0), (2, 0, 1), (1, 1, 1)\}$



Ejemplo – Cruce de río

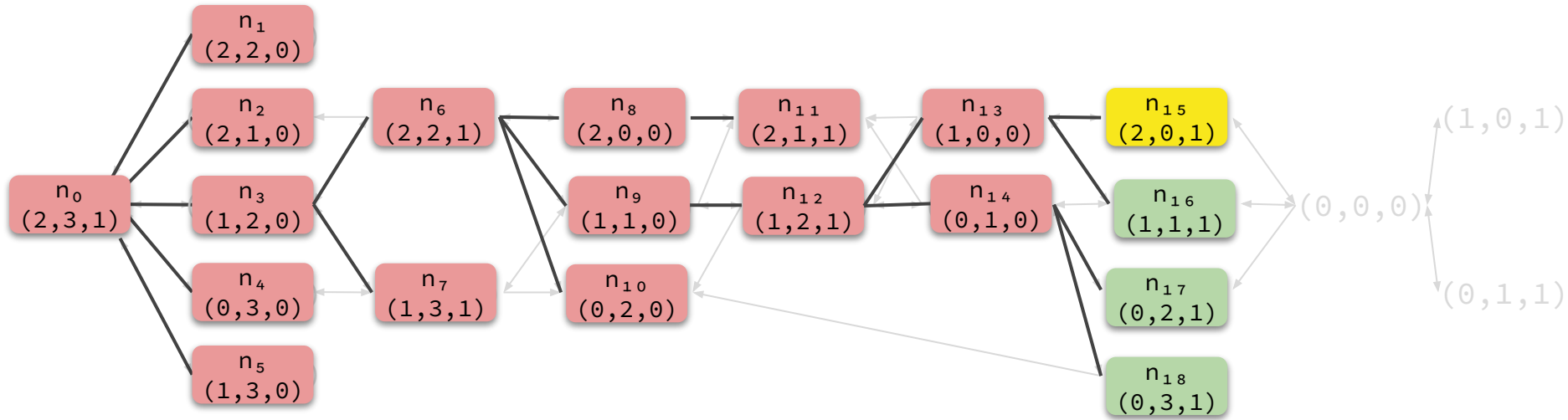


FRONTERA = $\{n_{15}, n_{16}, n_{17}, n_{18}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1)\}$



Ejemplo – Cruce de río

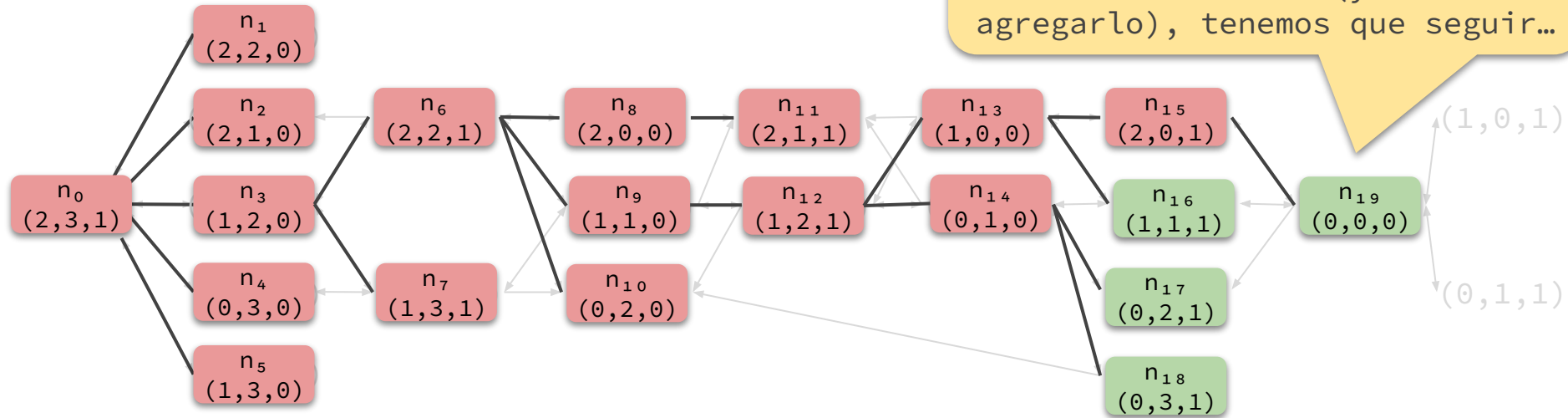


FRONTERA = $\{n_{16}, n_{17}, n_{18}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1)\}$



Ejemplo – Cruce de río

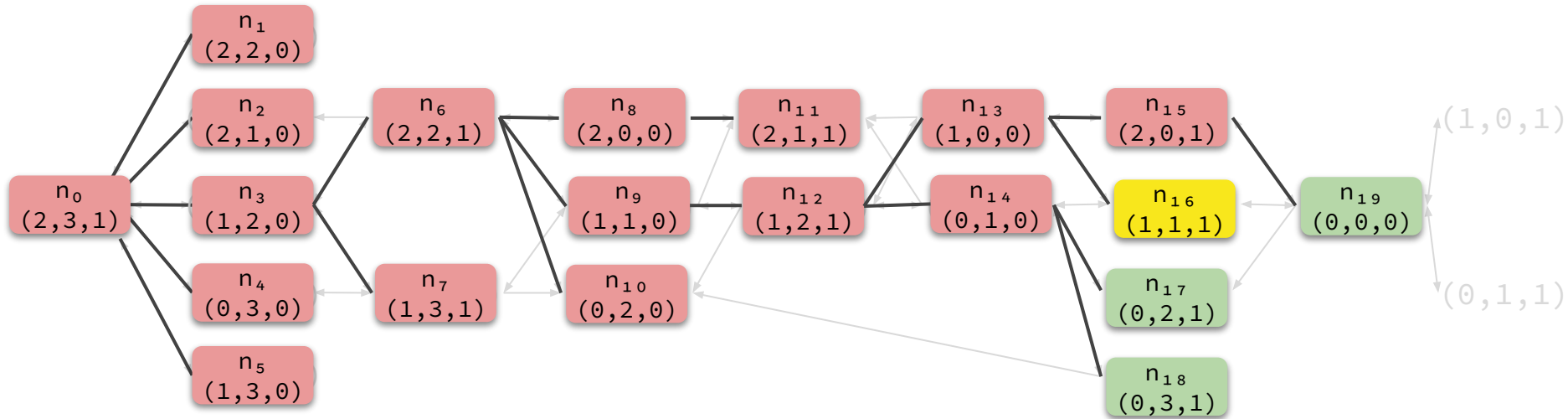


FRONTERA = $\{n_{16}, n_{17}, n_{18}, n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río

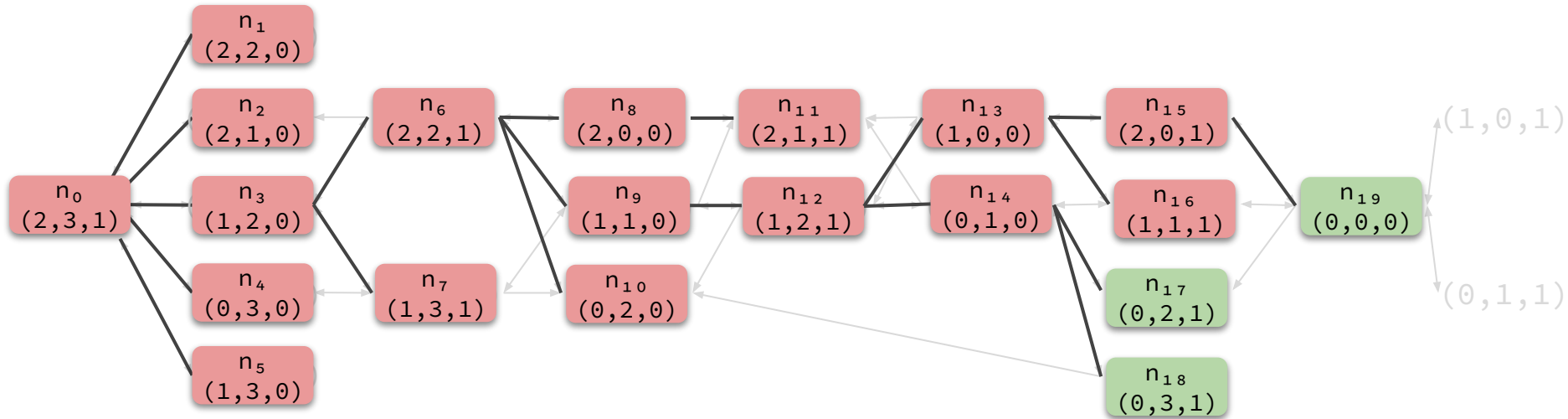


FRONTERA = $\{n_{17}, n_{18}, n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río

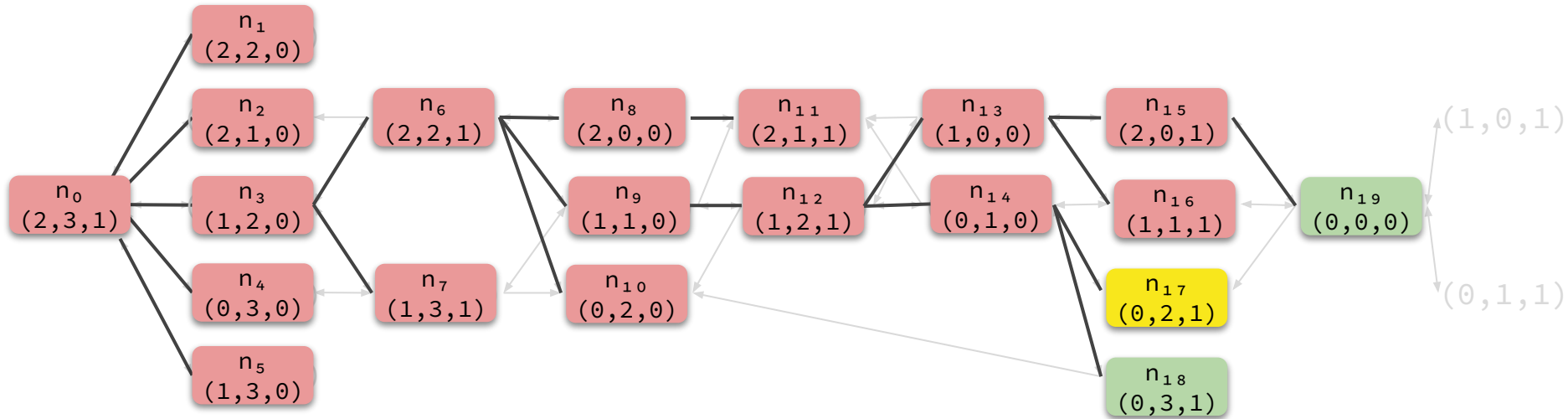


FRONTERA = $\{n_{17}, n_{18}, n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río

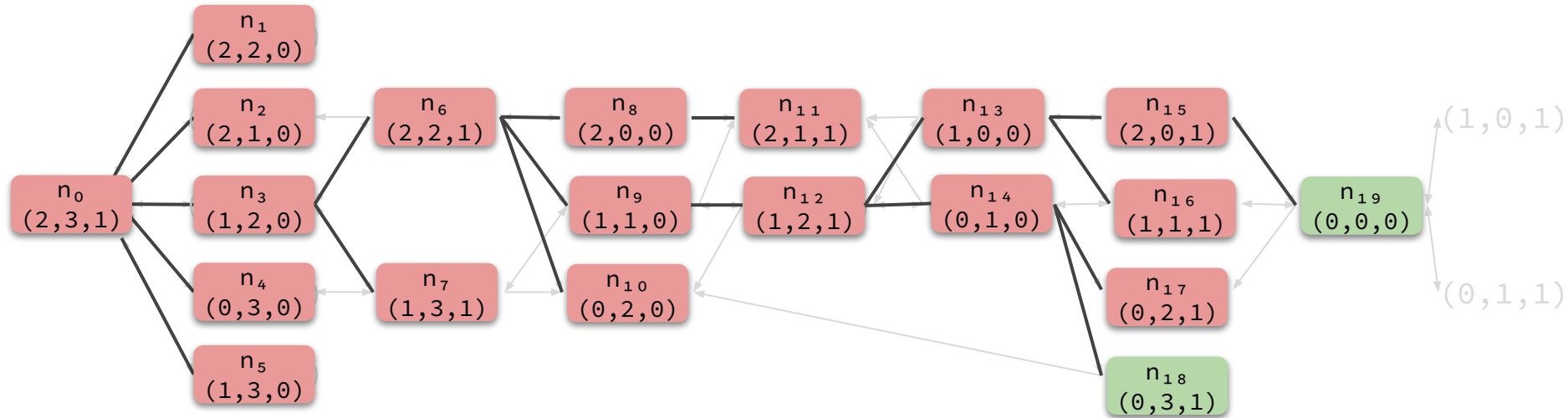


FRONTERA = $\{n_{18}, n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río

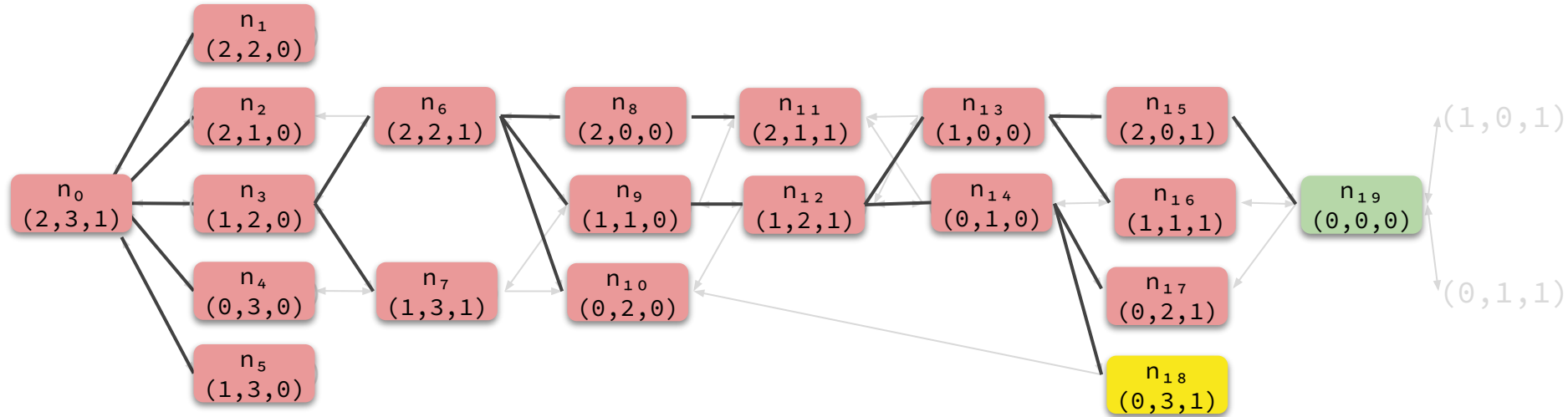


FRONTERA = $\{n_{18}, n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río

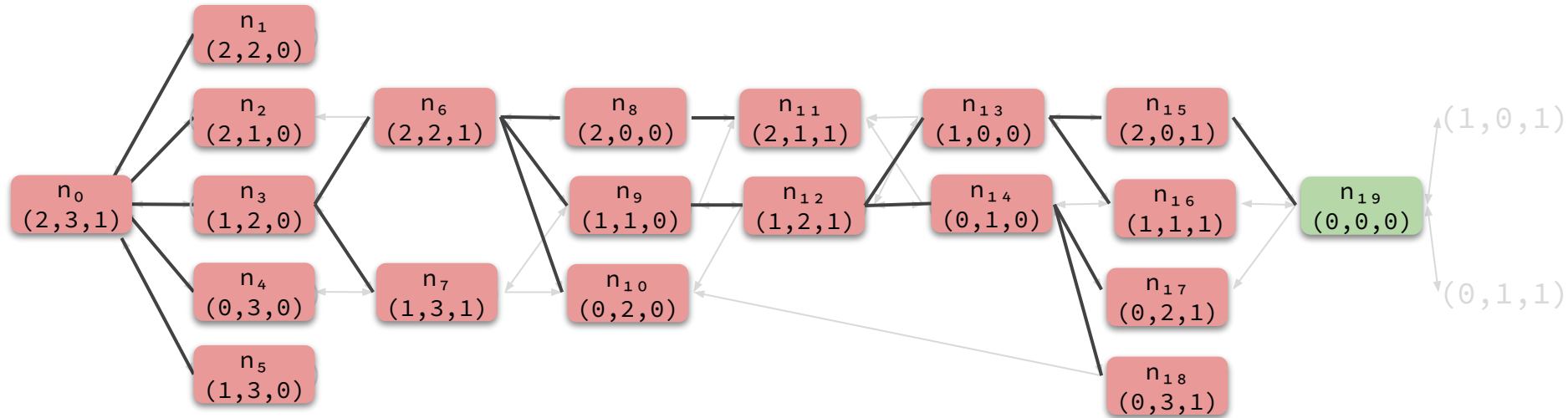


FRONTERA = $\{n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río



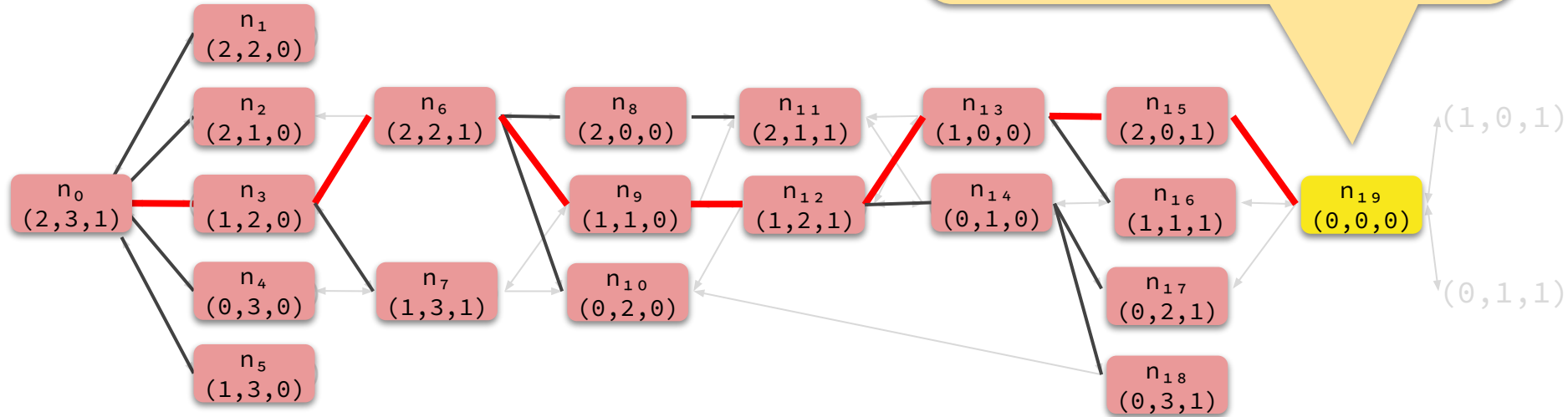
FRONTERA = $\{n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$



Ejemplo – Cruce de río

Encontramos un estado objetivo y retrocediendo por el camino hacia la raíz tenemos una solución.

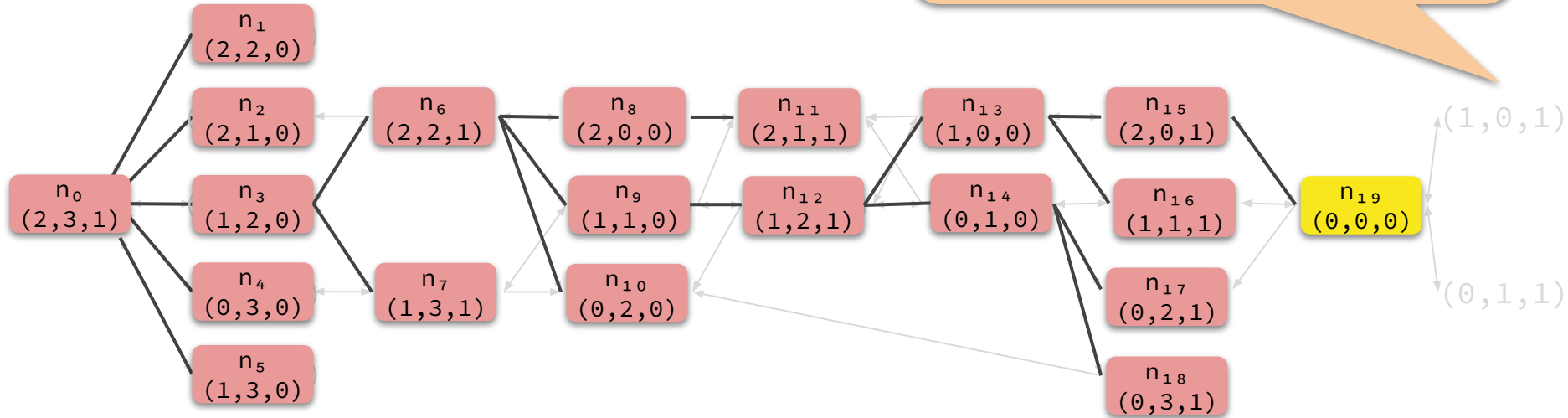


FRONTERA = {}

ALCANZADOS = {(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)}



En este caso, evitamos generar 2 de los 22 nodos del grafo de espacio de estados, es decir, un ahorro del **9%** de memoria y tiempo.

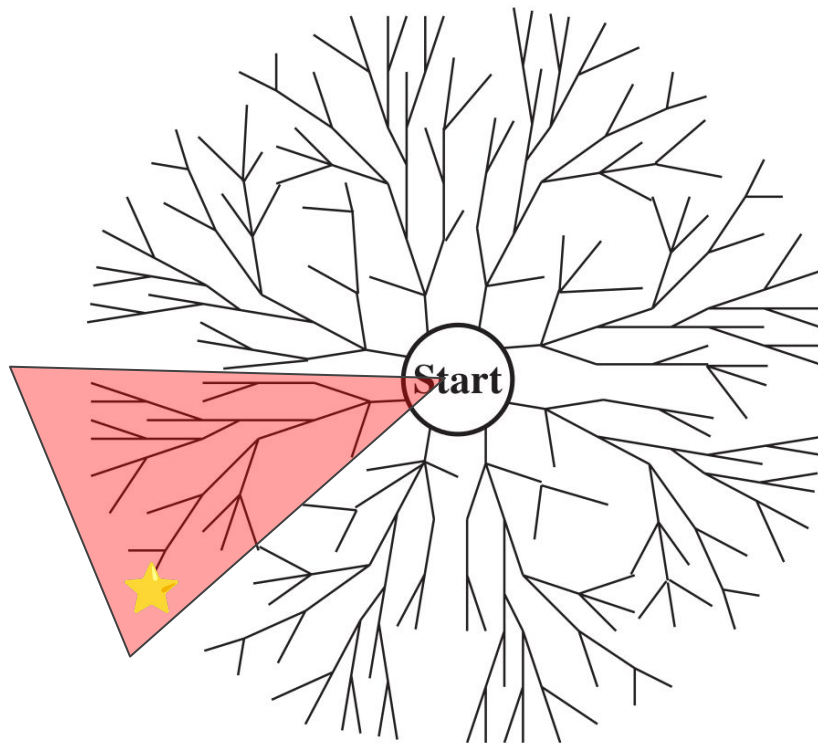


FRONTERA = { }

$$\text{ALCANZADOS} = \{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$$

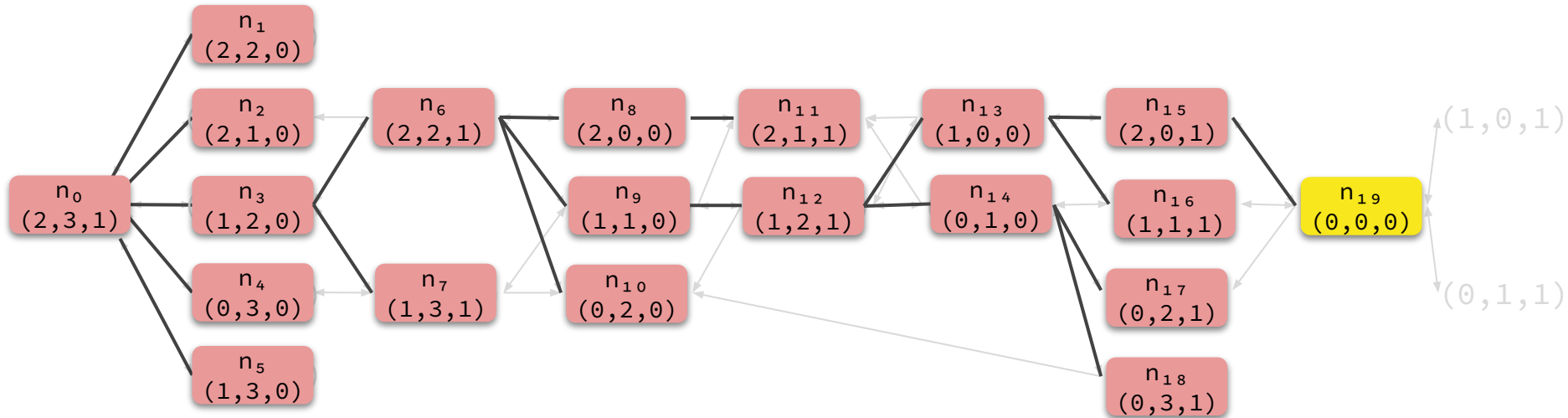
Eficiencia de la búsqueda

Un 9% de ahorro puede parecer insuficiente, pero en problemas con grandes espacios de estados, el ahorro puede ser **muchísimo mayor**.





Ejemplo – Cruce de río



La búsqueda de grafo construye dinámicamente un **árbol de búsqueda** dentro del grafo de estados.



Árbol de búsqueda

— — —

¿Qué estructura de datos podemos usar para almacenar un árbol de búsqueda? 🤔

- Cada **nodo** contiene un estado. En particular, la **raíz** contiene el estado inicial.
- De cada nodo (**padre**) y por cada acción, hay una **rama** que lo conecta con un nodo con el estado sucesor (**hijo**).
- Una **hoja** es un nodo sin hijos (estados sin sucesores).
- El **nivel** o **profundidad** de un nodo es el largo del camino que conecta ese nodo con la raíz.

Implementación

— — —

```
estado = ...  
padre  = ...  
acción = ...  
costo  = ...
```

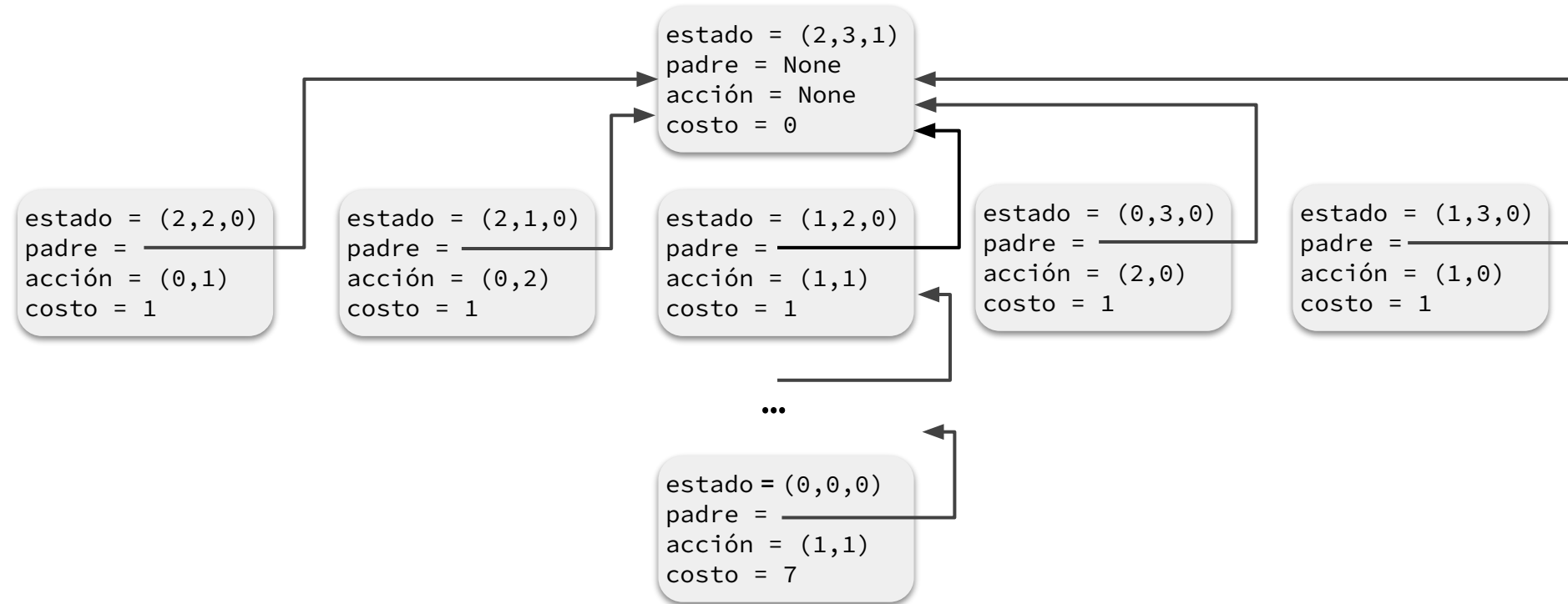
Un nodo n se puede representar con una estructura con 4 (o 5) atributos:

- **$n.estado$** : estado almacenado.
- **$n.padre$** : nodo padre.
- **$n.acción$** : acción aplicada en el padre.
- **$n.costo$** : costo de camino de la raíz a n .
- **$n.profundidad$** (opcional): profundidad de n .



Ejemplo – Cruce de río

— — —



Implementación

— — —

- En Prog2, cada nodo guardaba referencias a sus hijos.
- En Prog3, cada nodo guarda una referencia a su padre.
¿Por qué? 🤔

Implementación

— — —

- En Prog2, cada nodo guardaba referencias a sus hijos.
- En Prog3, cada nodo guarda una referencia a su padre.
¿Por qué? 🤔



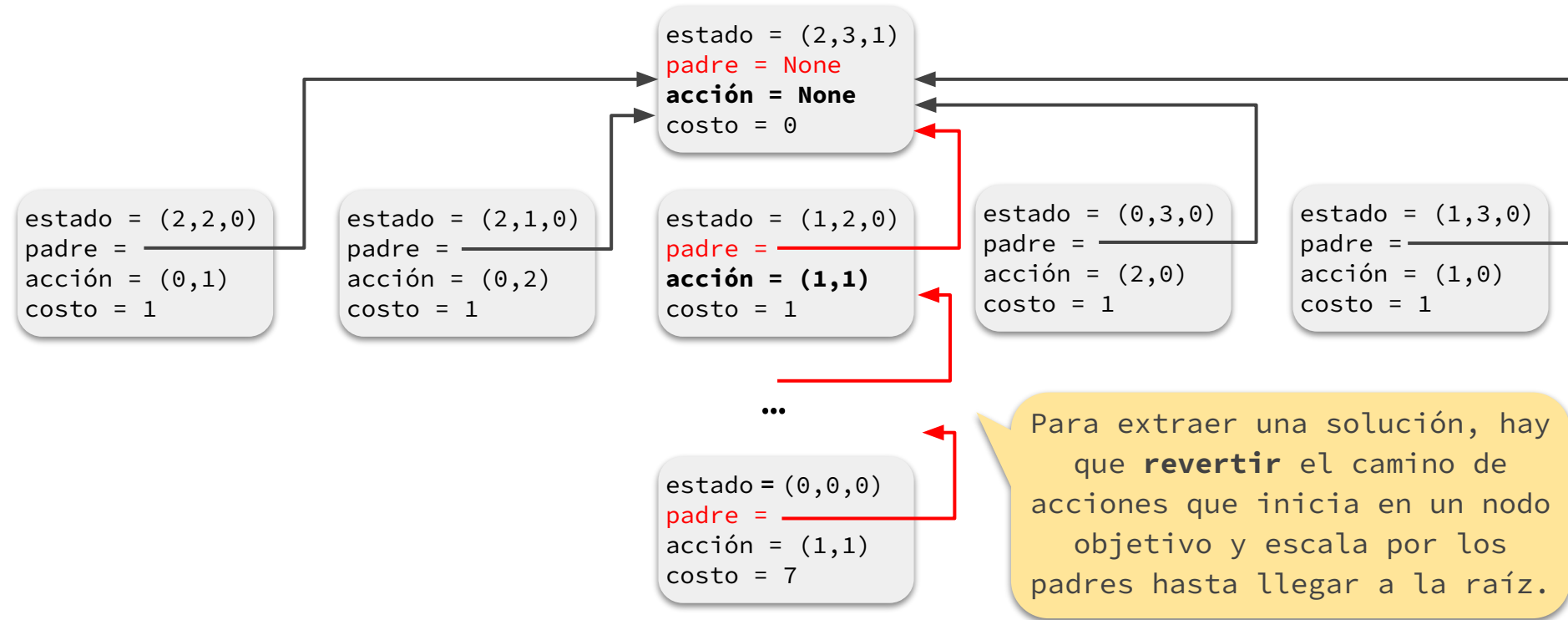
Ventaja. Podemos recuperar fácilmente una solución.

Una vez encontrado un nodo objetivo, retrocedemos por los padres hasta la raíz.



Ejemplo – Cruce de río

— — —





Algoritmo general de búsqueda de grafo

```
1 function GRAPH-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   ALCANZADOS  $\leftarrow \{n_0.\text{estado}\}$ 
4   FRONTERA  $\leftarrow \{n_0\}$ 
5   do
6     if FRONTERA.empty() then return no-solución
7      $n \leftarrow \text{FRONTERA.pop}()$ 
8     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
9     for all  $a$  in problema.acciones( $n.\text{estado}$ ) do
10        $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
11       if  $s'$  not in ALCANZADOS then
12         ALCANZADOS.insert( $s'$ )
13          $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
14         FRONTERA.push( $n'$ )
```



Algoritmo general de búsqueda de grafo

```
1 function GRAPH-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   ALCANZADOS  $\leftarrow \{n_0.\text{estado}\}$ 
4   FRONTERA  $\leftarrow \{n_0\}$ 
5   do
6     if FRONTERA.empty() then return no-solución
7      $n \leftarrow \text{FRONTERA.pop}()$ 
8     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
9     for all  $a$  in problema.acciones( $n.\text{estado}$ ) do
10       $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
11      if  $s' \text{ not in ALCANZADOS}$  then
12        ALCANZADOS.insert( $s'$ )
13         $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
14        FRONTERA.push( $n'$ )
```

¿Qué ED elegimos para ALCANZADOS?

Un **conjunto** o **diccionario**.

¿Qué ED elegimos para FRONTERA?

Lo veremos próximamente.

Observar que **GRAPH-SEARCH** puede aplicarse a cualquier problema de búsqueda, por eso decimos que es **general**.

Eficiencia de GRAPH-SEARCH

Notar que siempre que se agrega un nodo a FRONTERA, su estado también se agrega a ALCANZADOS.

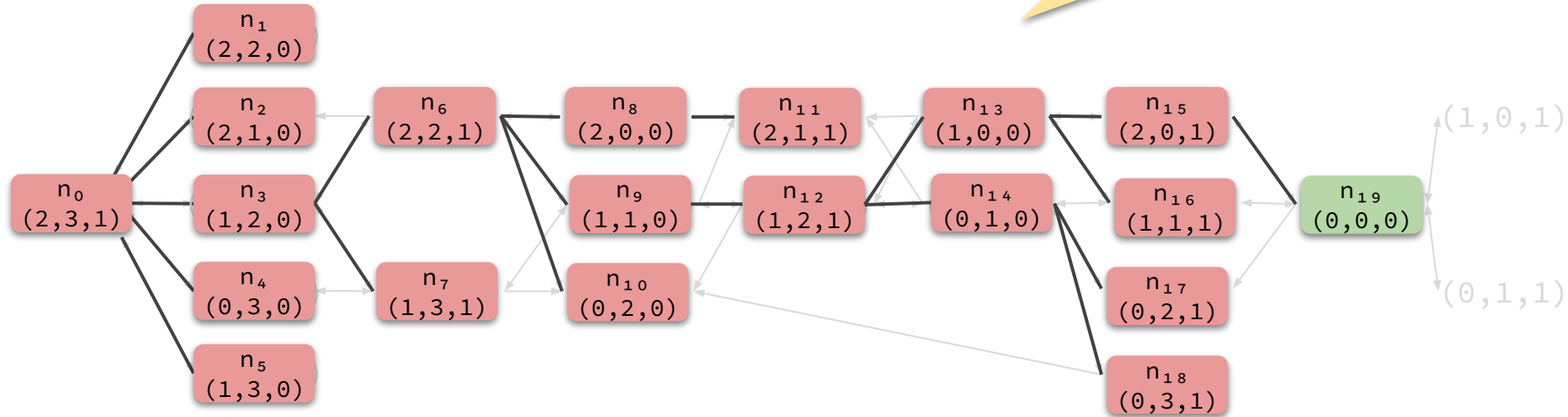
Por lo tanto, el consumo de memoria de GRAPH-SEARCH depende principalmente de:

El máximo número de estados en ALCANZADOS.



Ejemplo – Cruce de río

El mayor número de estados en ALCANZADOS se da en esta iteración, siendo = **20**.



FRONTERA = $\{n_{19}\}$

ALCANZADOS = $\{(2,3,1), (2,2,0), (2,1,0), (1,2,0), (0,3,0), (1,3,0), (2,2,1), (1,3,1), (2,0,0), (1,1,0), (0,2,0), (2,1,1), (1,2,1), (1,0,0), (0,1,0), (2,0,1), (1,1,1), (0,2,1), (0,3,1), (0,0,0)\}$

Eficiencia de GRAPH-SEARCH

- El consumo de memoria de GRAPH-SEARCH puede volverse **prohibitivo**.
 - En el peor de los casos, **ALCANZADOS podría contener tantos estados como estados alcanzables tenga el problema**.
- ¿Es realmente necesario mantener los estados alcanzados en memoria? ¿Qué pasaría si no lo hacemos? 🤔

Búsqueda de árbol

Búsqueda de árbol

La **búsqueda de árbol** es similar a la búsqueda de grafo, salvo que **no se mantiene en memoria un conjunto con los estados alcanzados previamente**.

Por supuesto, el árbol de búsqueda que se construye podría contener nodos con estados **repetidos**.

Búsqueda de árbol

1. **Inicialización.** Generar la raíz n_0 con el estado inicial s_0 . Poner ~~en ALCANZADOS a s_0 y~~ en FRONTERA a n_0 .
2. **Iteración.** Mientras FRONTERA no esté vacía:
3. **Selección.** Sacar un nodo n de FRONTERA.
4. **Test objetivo.** Retornar si n es objetivo.
5. **Expansión.** Expandir n y actualizar FRONTERA $\forall$ ~~ALCANZADOS~~ con sus vecinos ~~no alcanzados~~.



Ejemplo – Cruce de río

— — —

n_0
(2, 3, 1)

FRONTERA = $\{n_0\}$



Ejemplo – Cruce de río

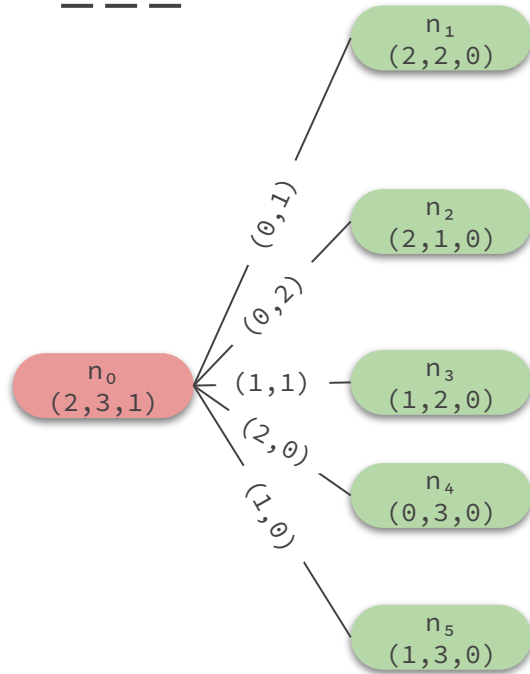
— — —

n_0
(2, 3, 1)

FRONTERA = {}



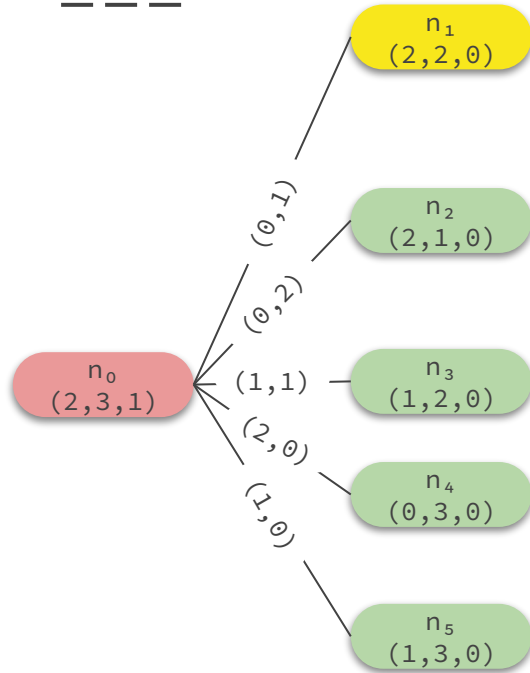
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_5\}$



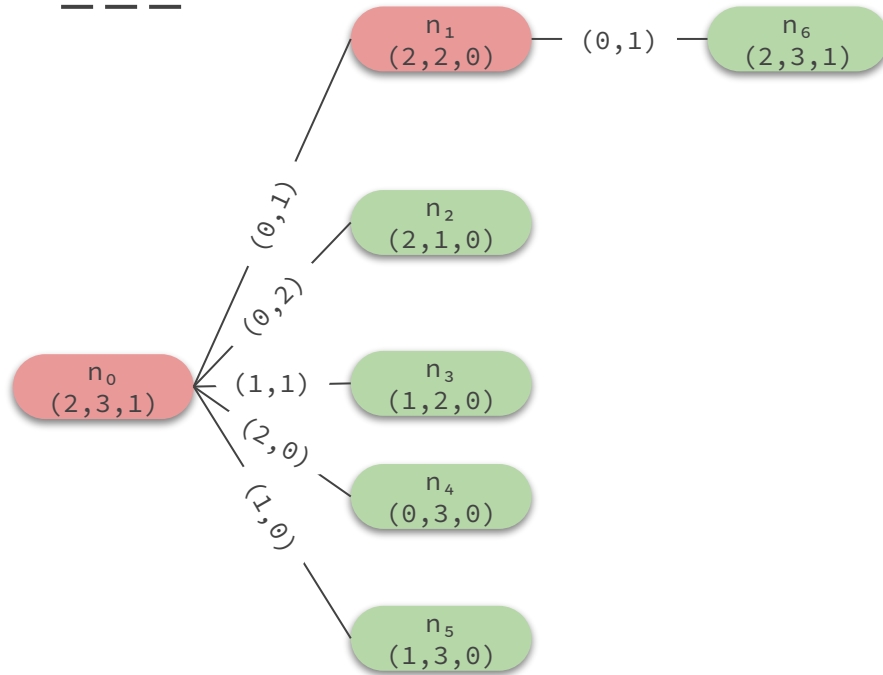
Ejemplo – Cruce de río



FRONTERA = $\{n_2, n_3, n_4, n_5\}$



Ejemplo – Cruce de río

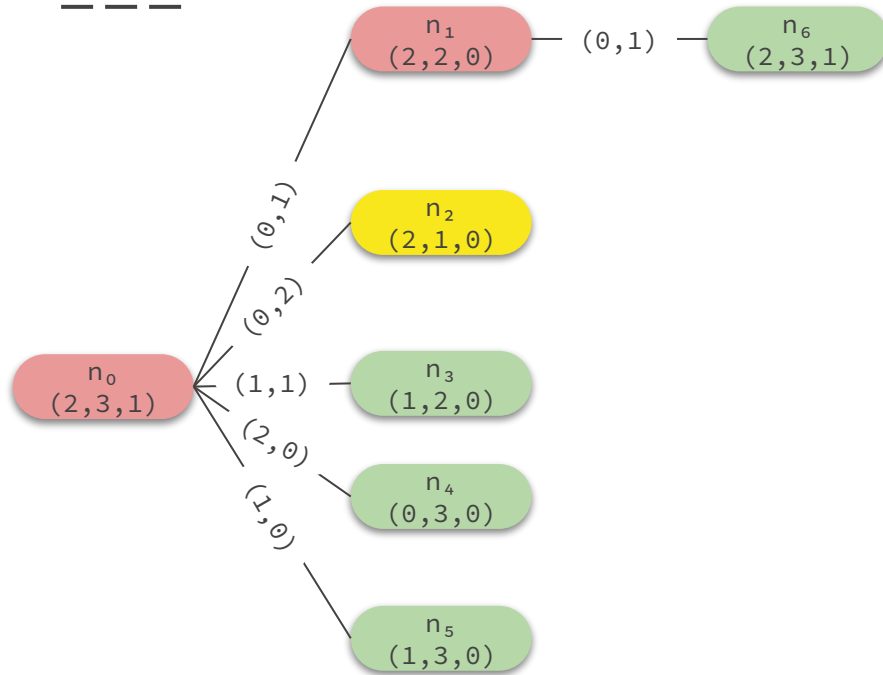


Los nodos n_0 y n_6 repiten el estado $(2, 3, 1)$.

FRONTERA = $\{n_2, n_3, n_4, n_5, n_6\}$



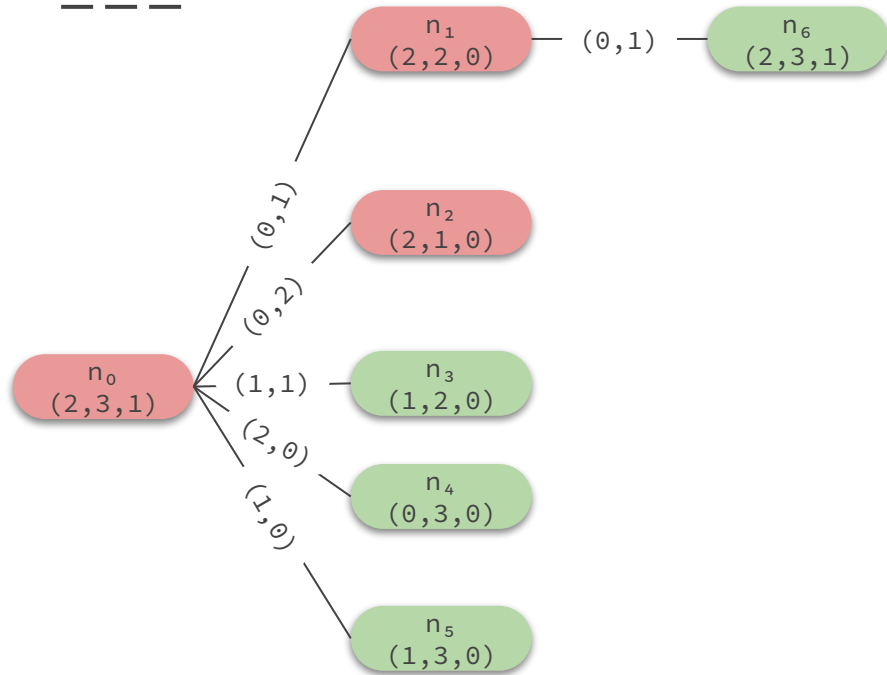
Ejemplo – Cruce de río



FRONTERA = $\{n_3, n_4, n_5, n_6\}$



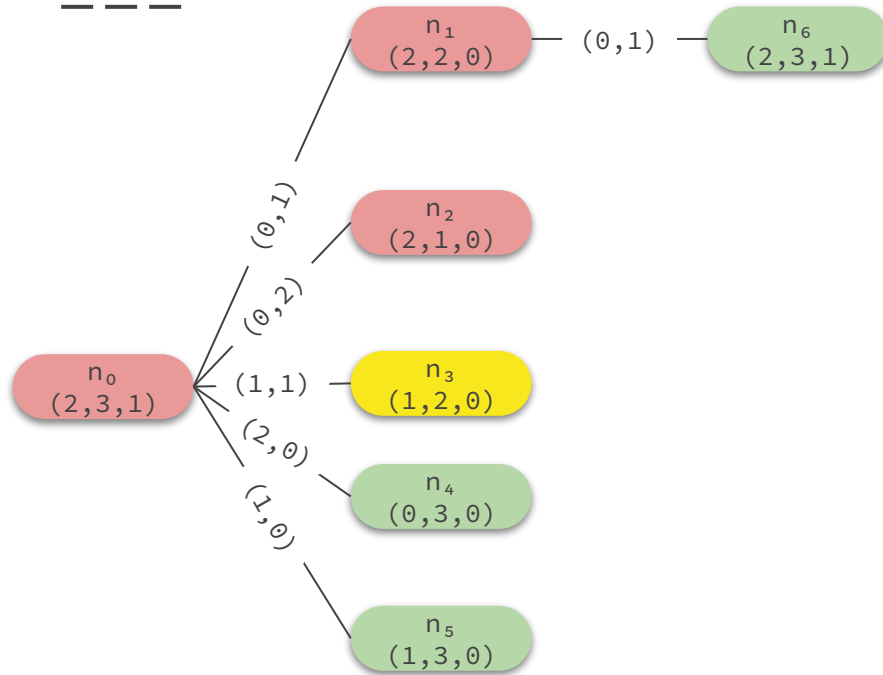
Ejemplo – Cruce de río



FRONTERA = $\{n_3, n_4, n_5, n_6\}$



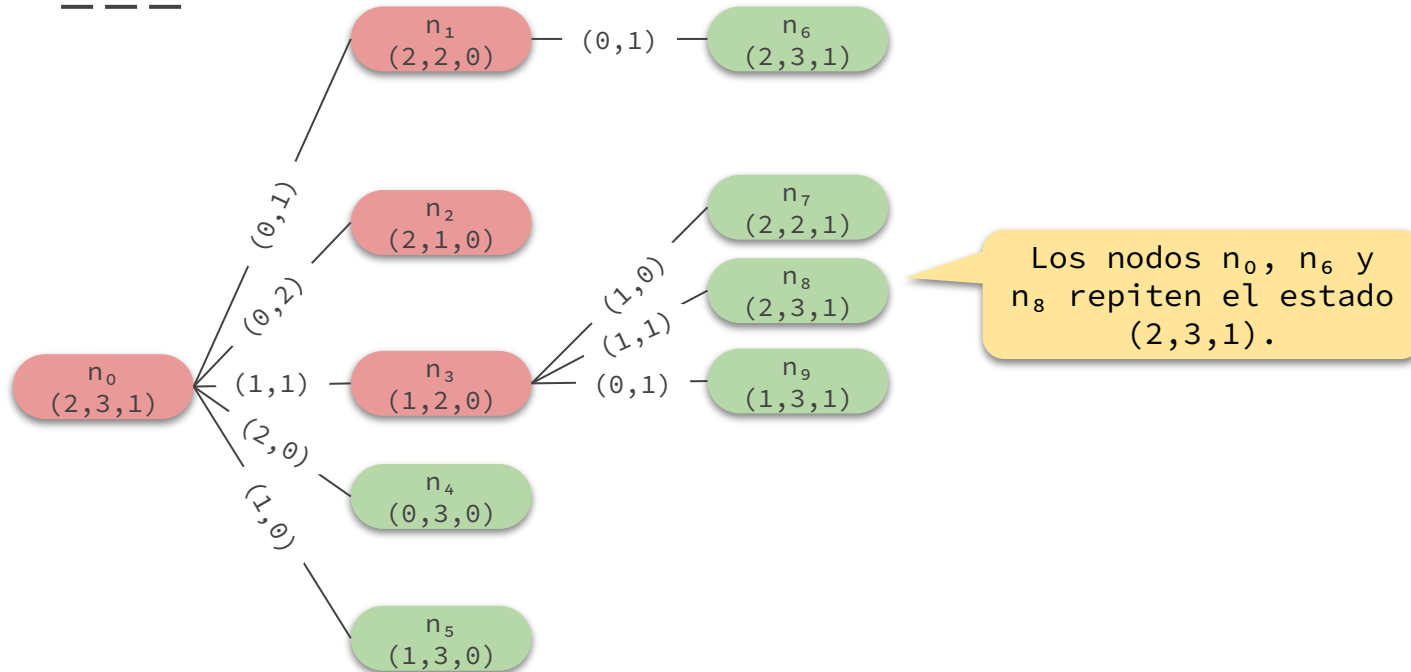
Ejemplo – Cruce de río



FRONTERA = $\{n_4, n_5, n_6\}$



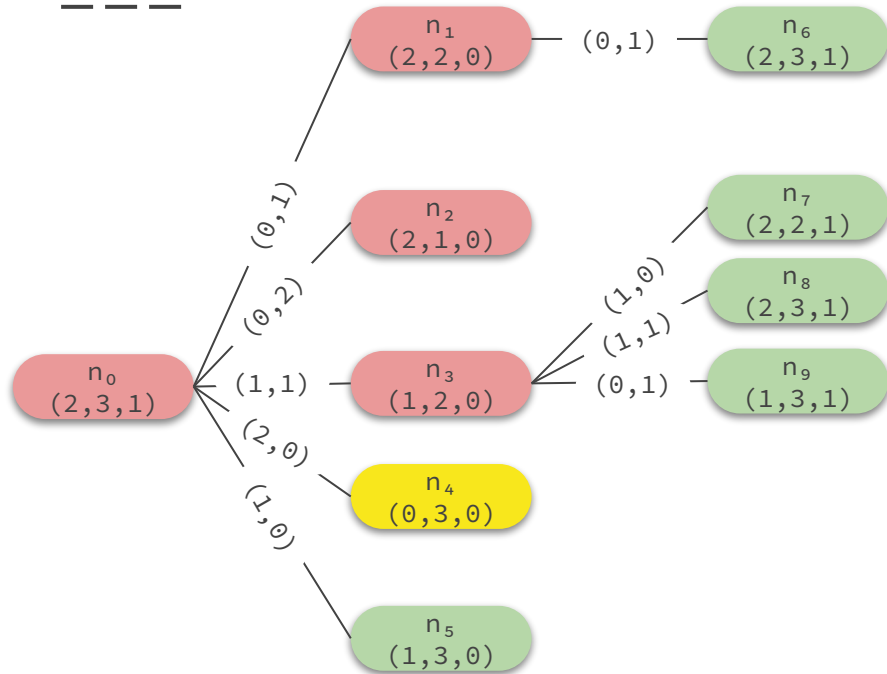
Ejemplo – Cruce de río



FRONTERA = $\{n_4, n_5, n_6, n_7, n_8, n_9\}$



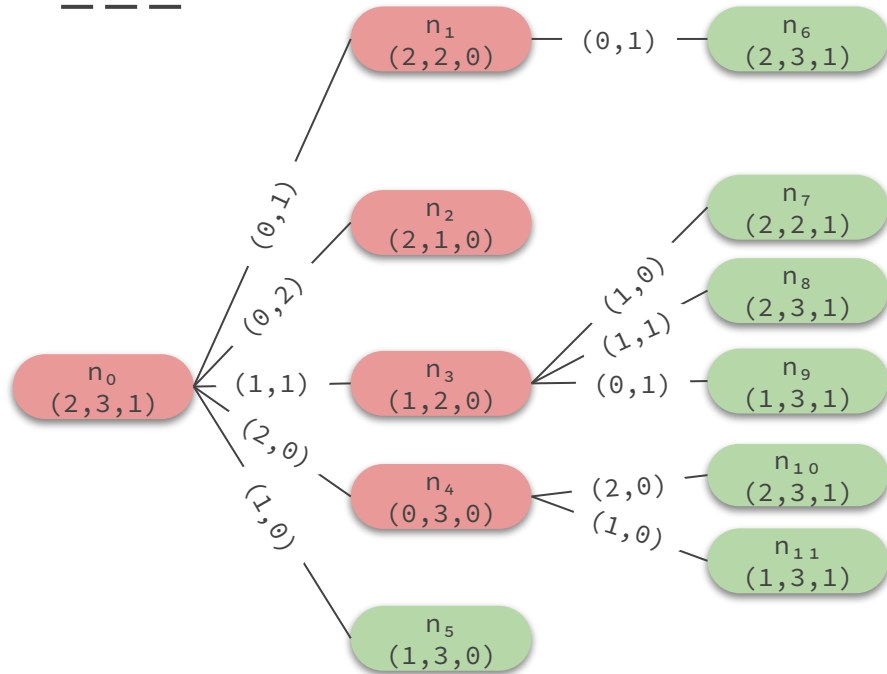
Ejemplo – Cruce de río



FRONTERA = $\{n_5, n_6, n_7, n_8, n_9\}$



Ejemplo – Cruce de río

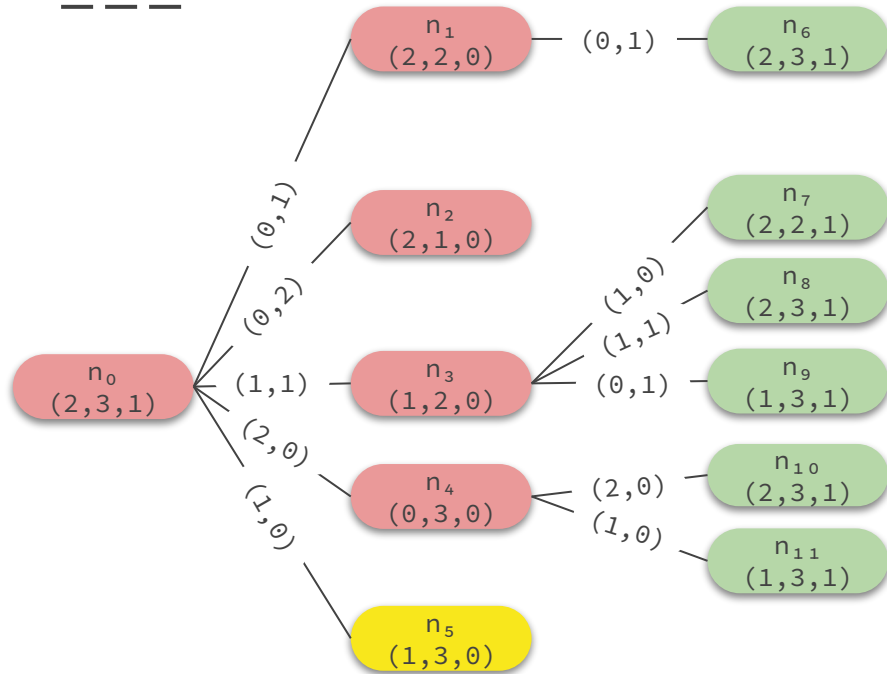


Los nodos n_0 , n_6 , n_8 y n_{10} repiten el estado $(2, 3, 1)$. Los nodos n_9 y n_{11} repiten el estado $(1, 3, 1)$.

FRONTERA = $\{n_5, n_6, n_7, n_8, n_9, n_{10}, n_{11}\}$



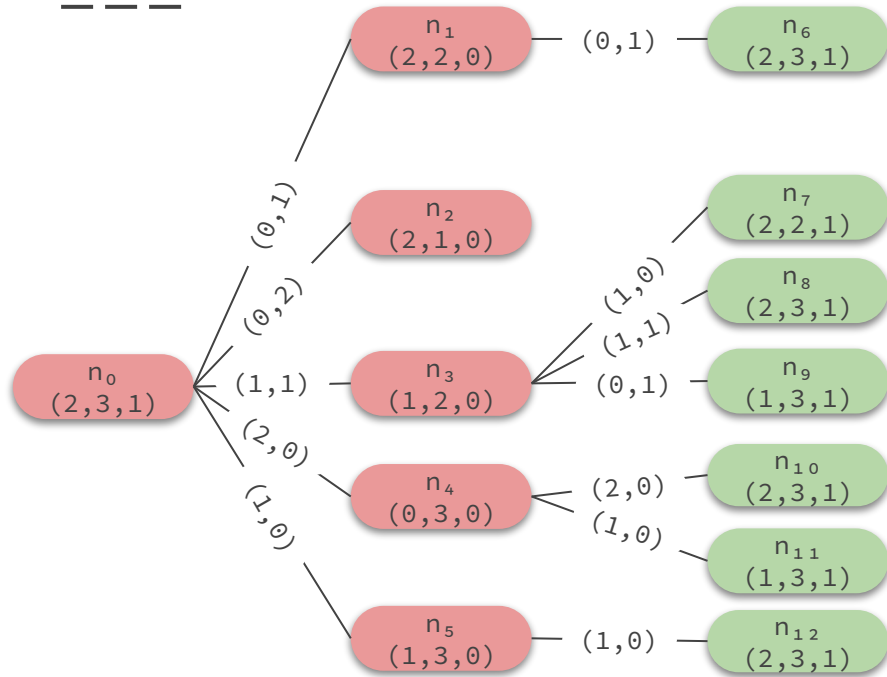
Ejemplo – Cruce de río



FRONTERA = $\{n_6, n_7, n_8, n_9, n_{10}, n_{11}\}$



Ejemplo – Cruce de río

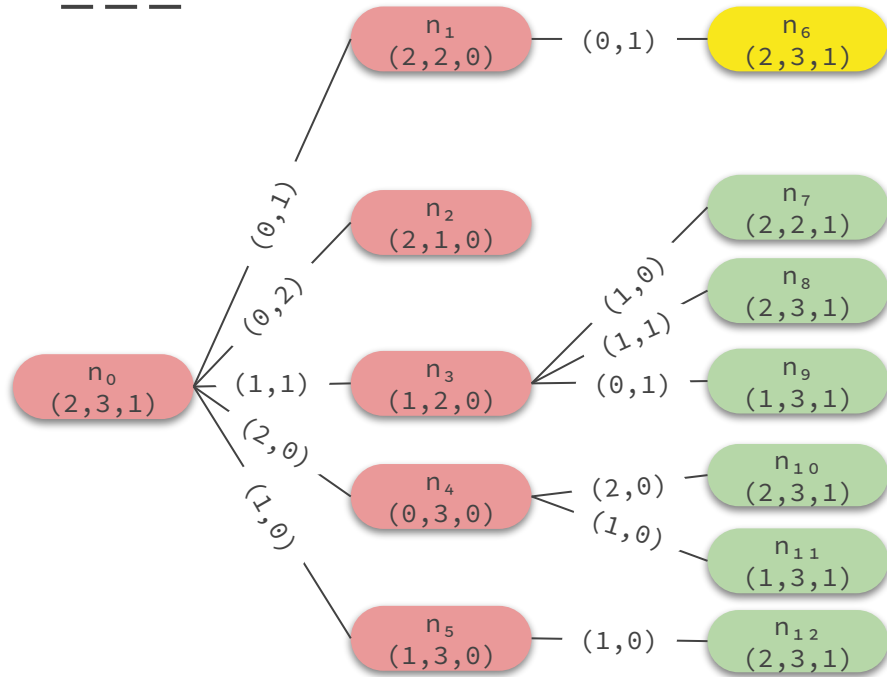


Los nodos n_0 , n_6 , n_8 , n_{10} y n_{12} repiten el estado $(2, 3, 1)$.

FRONTERA = $\{n_6, n_7, n_8, n_9, n_{10}, n_{11}, n_{12}\}$



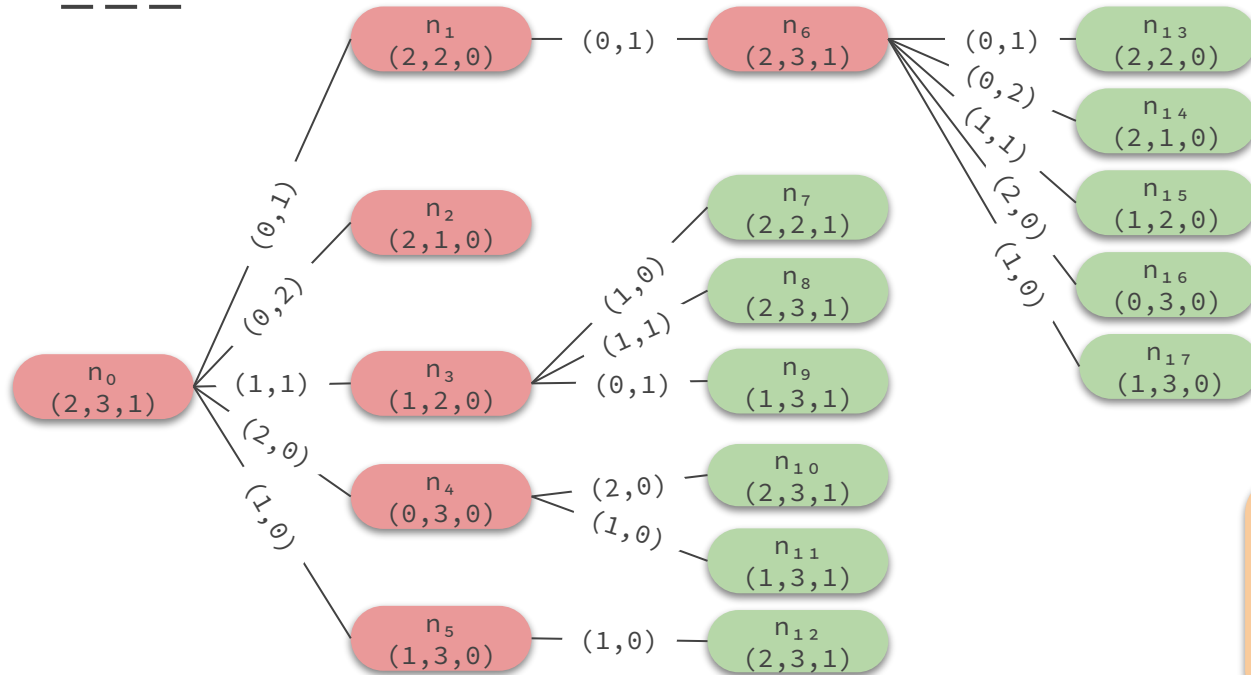
Ejemplo – Cruce de río



FRONTERA = $\{n_7, n_8, n_9, n_{10}, n_{11}, n_{12}\}$



Ejemplo – Cruce de río



Se vuelven a repetir todos los estados de los nodos del nivel 1.

Nosotros cortaremos el ejemplo acá, pero la búsqueda de árbol prosigue hasta encontrar una solución o la frontera se vacíe.

FRONTERA = {n₇, n₈, n₉, n₁₀, n₁₁, n₁₂, n₁₃, n₁₄, n₁₅, n₁₆, n₁₇}



Algoritmo general de búsqueda de árbol

— — —

```
1 function TREE-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   FRONTERA  $\leftarrow \{n_0\}$ 
4   do
5     if FRONTERA.empty() then return no-solución
6      $n \leftarrow \text{FRONTERA.pop}()$ 
7     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
8     for all  $a$  in problema.acciones( $n.\text{estado}$ ) do
9        $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
10       $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
11      FRONTERA.push( $n'$ )
```



Algoritmo general de búsqueda de árbol

```
1 function TREE-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   FRONTERA  $\leftarrow \{n_0\}$ 
4   do
5     if FRONTERA.empty() then return no-solución
6      $n \leftarrow \text{FRONTERA.pop}()$ 
7     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
8     for all  $a$  in problema.acciones( $n.\text{estado}$ ) do
9        $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
10       $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
11      FRONTERA.push( $n'$ )
```

¿Garantiza terminación? 🤔

¿Cuánta memoria consume? ¿Más o menos que GRAPH-SEARCH? 🤔

Vamos de a poco...



Ejemplo – Cruce de río

— — —

n_0
(2,3,1)

Vamos a repetir la búsqueda de árbol usando un criterio diferente para elegir los nodos de la frontera.

Esta vez, siempre se expande el **último nodo** de la frontera.

FRONTERA = $\{n_0\}$



Ejemplo – Cruce de río

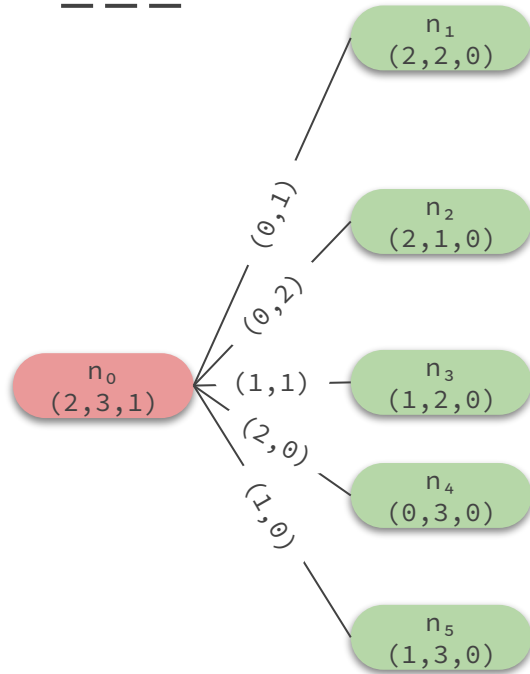
— — —

n_0
(2, 3, 1)

FRONTERA = {}



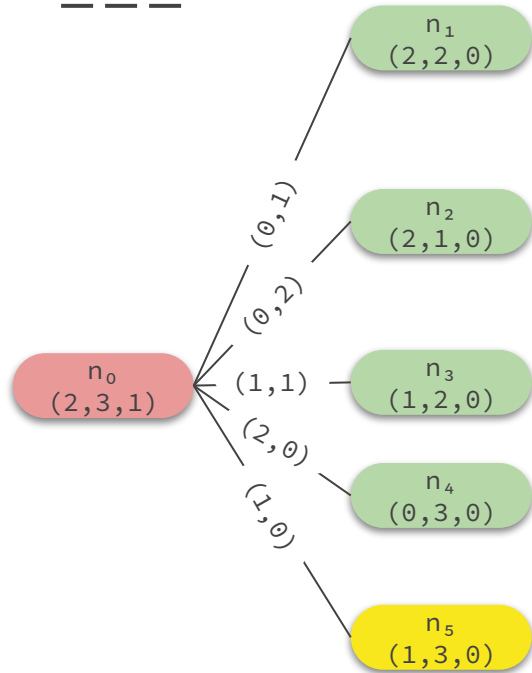
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_5\}$



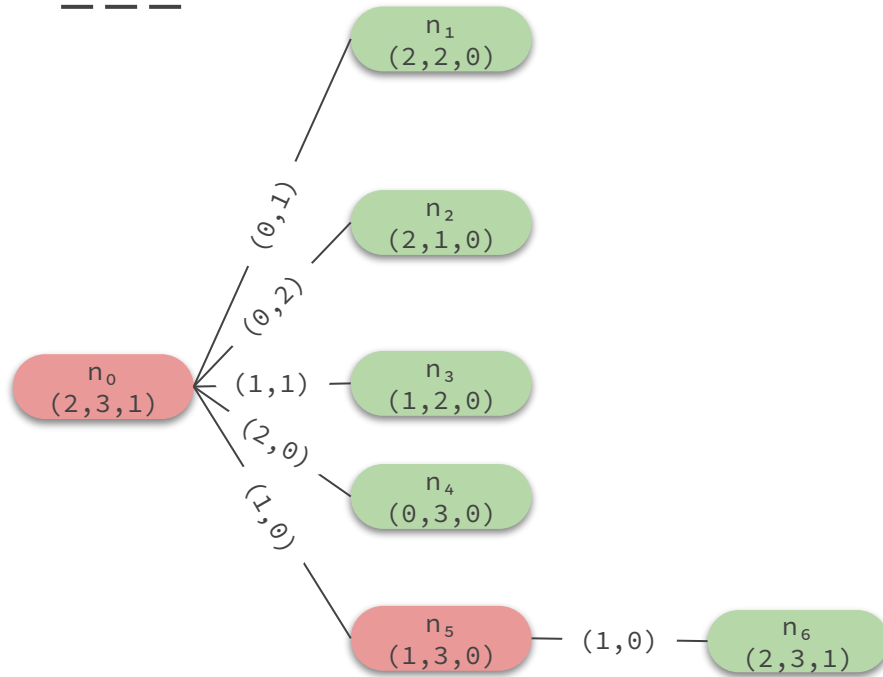
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4\}$



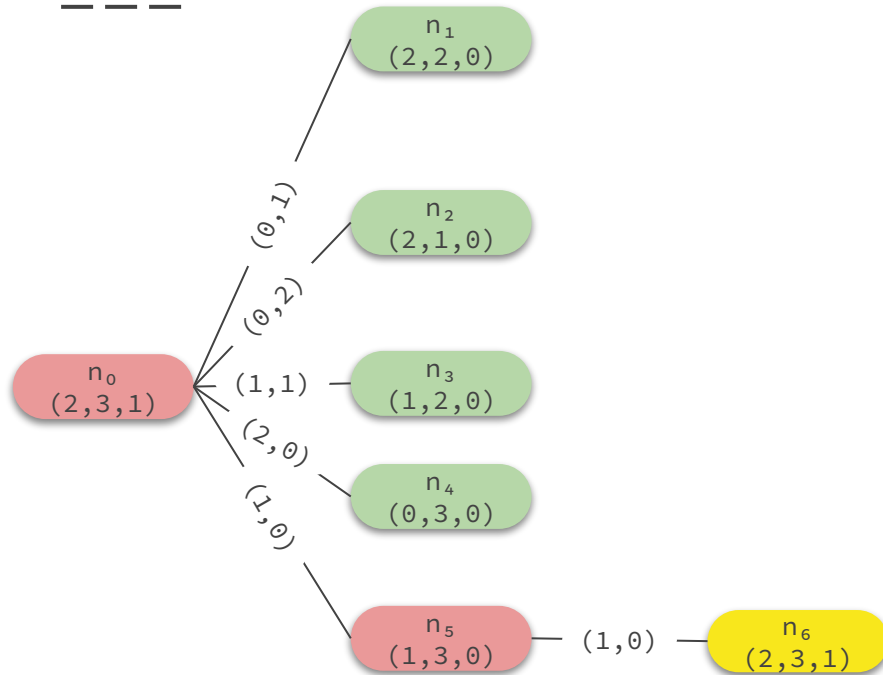
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_6\}$



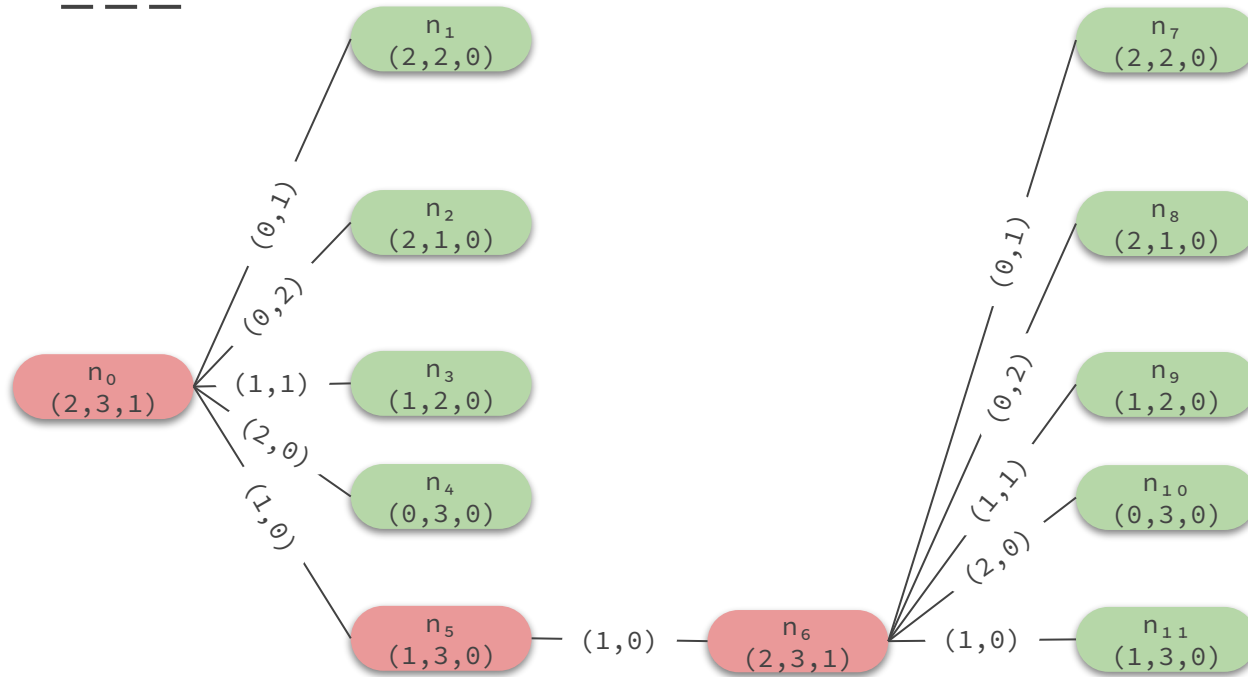
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4\}$



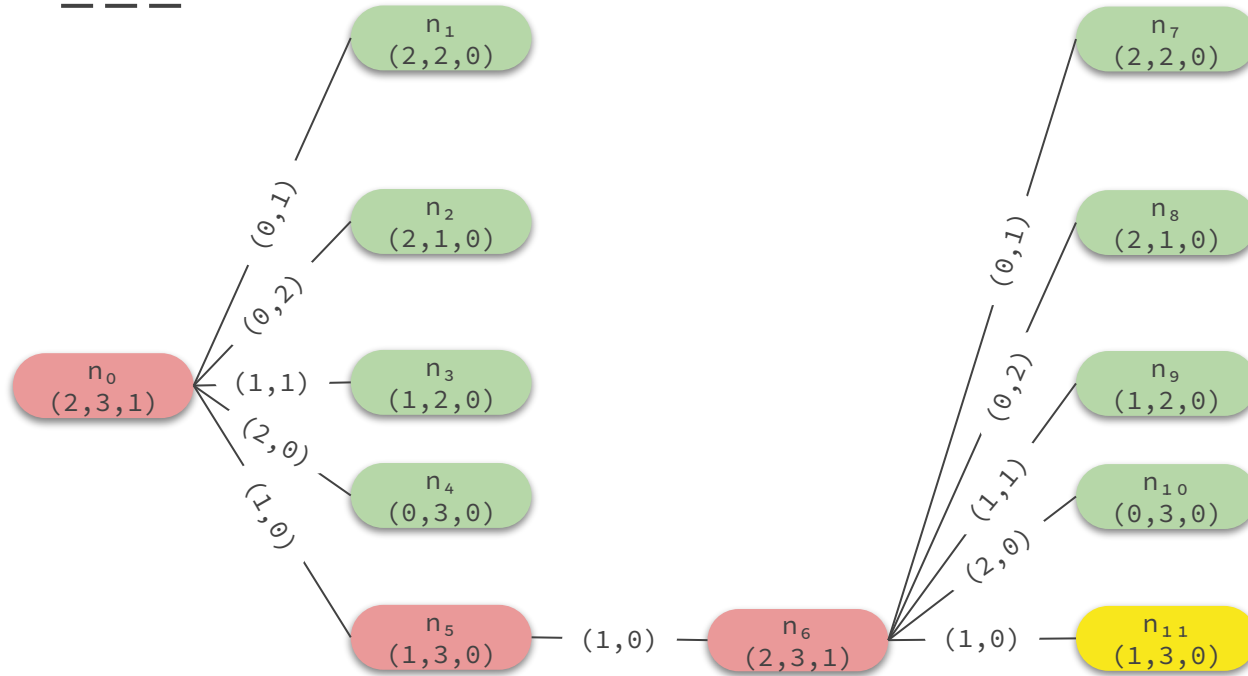
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_7, n_8, n_9, n_{10}, n_{11}\}$



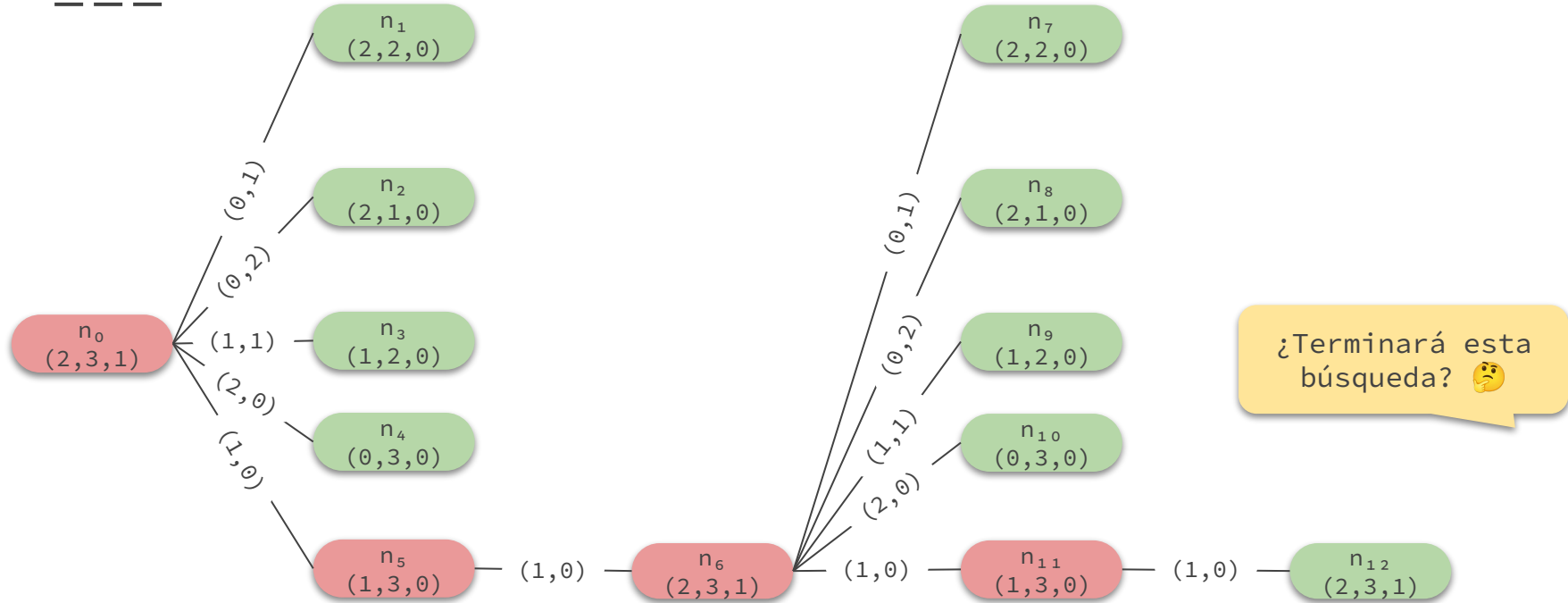
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_7, n_8, n_9, n_{10}\}$



Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_7, n_8, n_9, n_{10}, n_{12}\}$

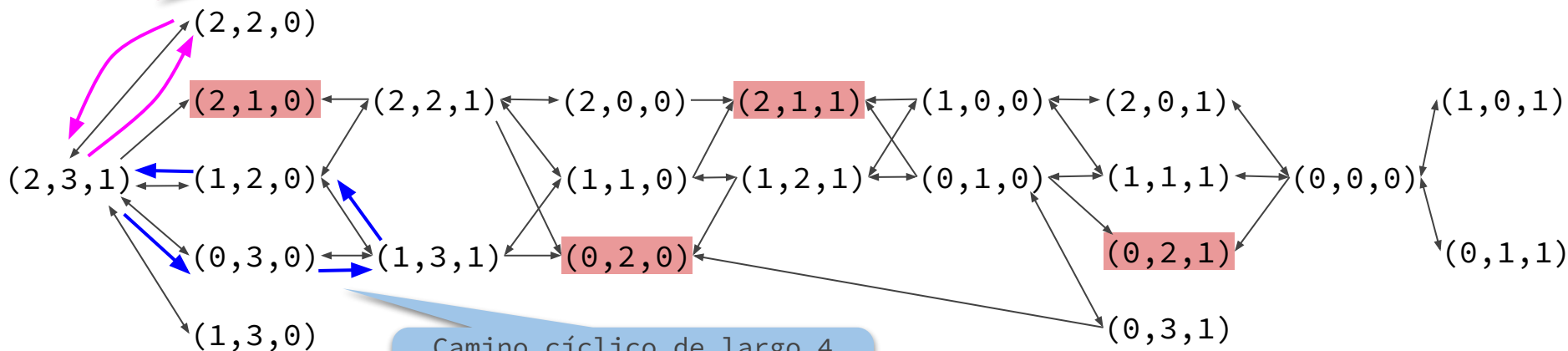
Camino cíclicos

- Un **camino cíclico** en el grafo de estados es un camino que pasa al menos dos veces por un mismo estado.
- En particular, un problema con **acciones reversibles** siempre tendrá caminos cíclicos.
- En presencia de caminos cíclicos, **el árbol de búsqueda podría ser infinito y TREE-SEARCH podría no terminar.**



Ejemplo – Cruce de río

Camino cíclico de largo 2
formado por una acción y la
acción que la revierte.



Camino cíclico de largo 4
donde no se revierten las
acciones.

¿Es posible detectar
caminos cíclicos
durante la búsqueda
de árbol? 🤔



Detección de caminos cíclicos

— — —

- Antes de agregar un nodo n a FRONTERA, controlamos **si el camino desde n a la raíz tiene estados repetidos**.
 - Si repite, **podamos** a n .
 - Sino, **agregamos** n a FRONTERA.
- Dado que cada nodo almacena a su padre, este control es fácil de agregar y no es demasiado costoso.
- **La detección de ciclos garantiza terminación en espacios de estados finitos.**



Ejemplo – Cruce de río

— — —

n_0
(2,3,1)

Vamos a repetir la
búsqueda de árbol
anterior incorporando la
detección de ciclos.

FRONTERA = $\{n_0\}$



Ejemplo – Cruce de río

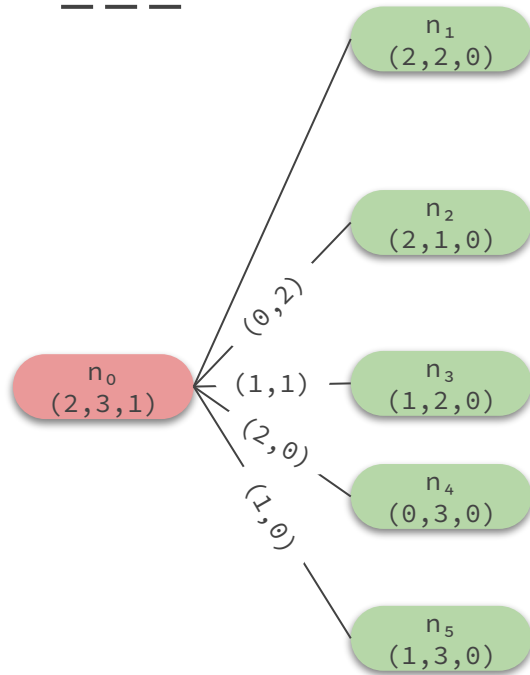
— — —

n_0
(2, 3, 1)

FRONTERA = {}



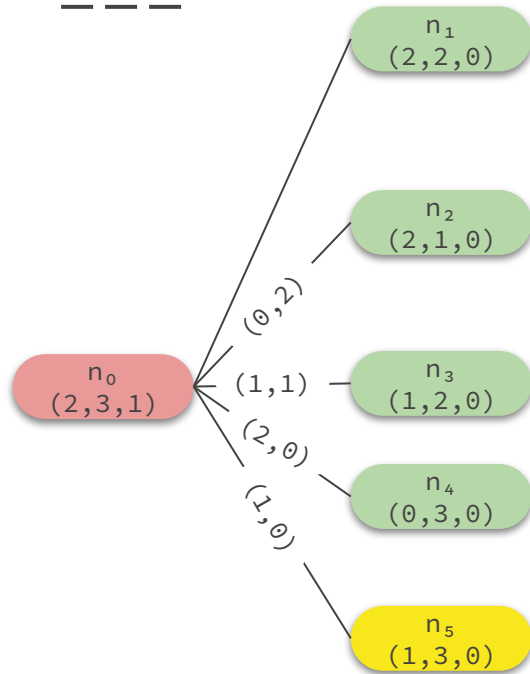
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_5\}$



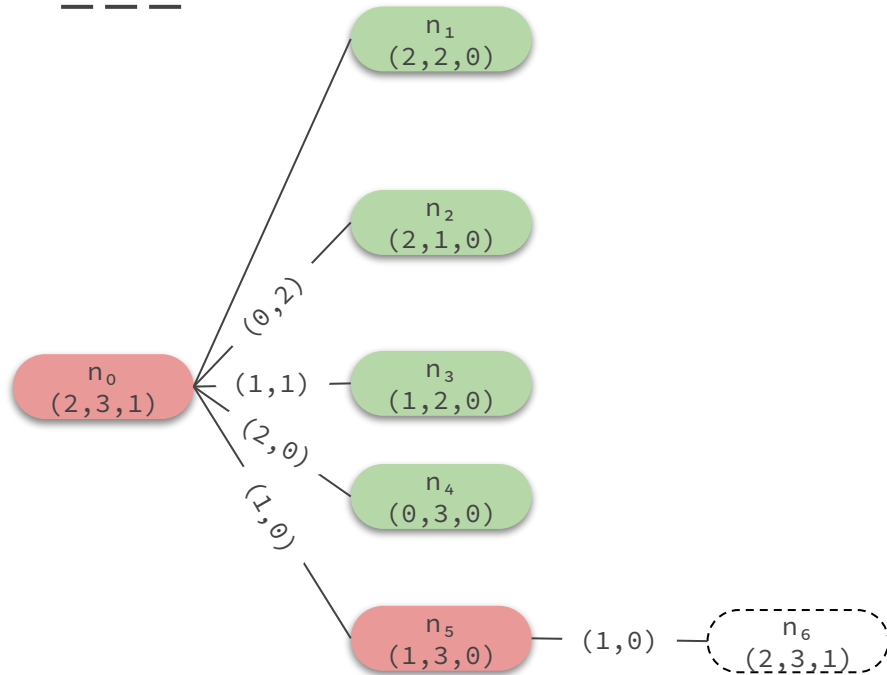
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3, n_4, n_5\}$



Ejemplo – Cruce de río

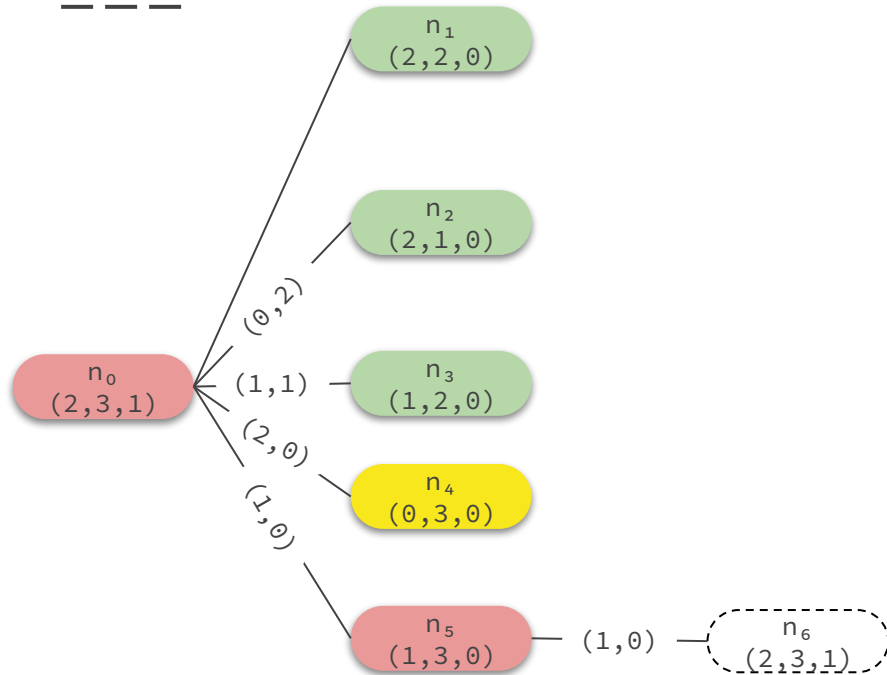


El nodo n_6 es **podado** porque repite el estado de n_0 .

FRONTERA = $\{n_1, n_2, n_3, n_4\}$



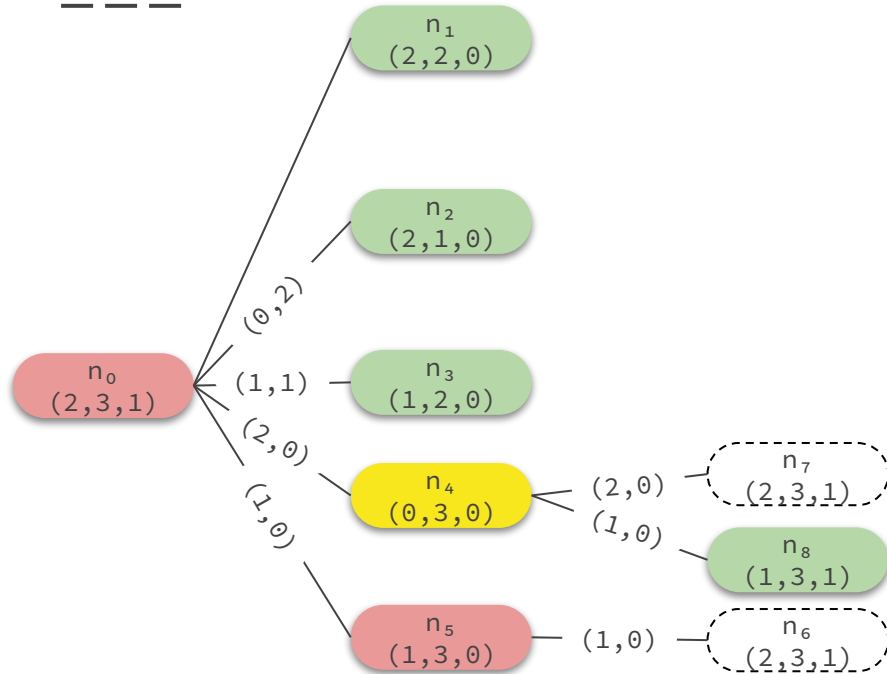
Ejemplo – Cruce de río



FRONTERA = $\{n_1, n_2, n_3\}$



Ejemplo – Cruce de río



Nosotros cortaremos el ejemplo acá, pero la búsqueda de árbol prosigue hasta encontrar una solución o la frontera se vacíe.

El nodo n_7 es **podado** porque repite el estado de n_0 .

FRONTERA = $\{n_1, n_2, n_3, n_8\}$



Algoritmo general de búsqueda de árbol con detección de ciclos

```
1 function TREE-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   FRONTERA  $\leftarrow \{n_0\}$ 
4   do
5     if FRONTERA.empty() then return no-solución
6      $n \leftarrow \text{FRONTERA.pop}()$ 
7     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
8     for all  $a$  in problema.acciones( $n.\text{estado}$ ) do
9        $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
10      if not hay_ciclo( $s', n$ ) then
11         $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
12        FRONTERA.push( $n'$ )
```

hay_ciclo(s', n): función que toma un estado s' y un nodo n y determina si s' se repite en el camino desde n a la raíz.

¿Será ésta la solución **definitiva** a la repetición de estados? 🤔



Ejemplo – Cruce de río

— — —

n_0
(2,3,1)

Vamos a repetir la
búsqueda de árbol con
detección de ciclos.

Esta vez, siempre
se expande el
primer nodo de la
frontera.

FRONTERA = $\{n_0\}$



Ejemplo – Cruce de río

— — —

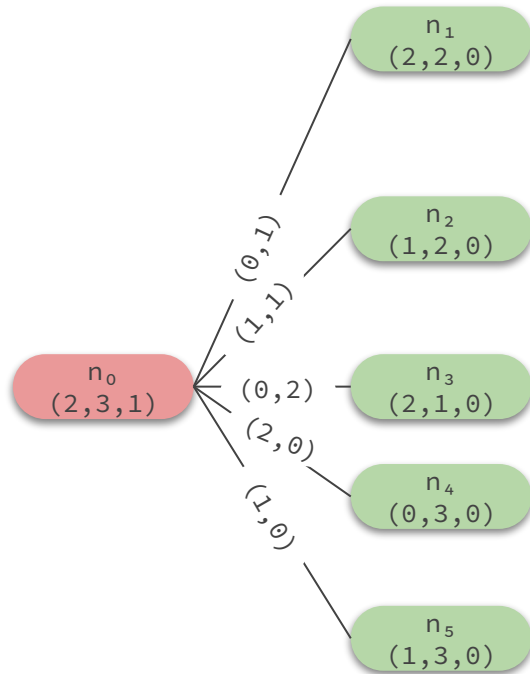
n_0
(2, 3, 1)

FRONTERA = {}



Ejemplo – Cruce de río

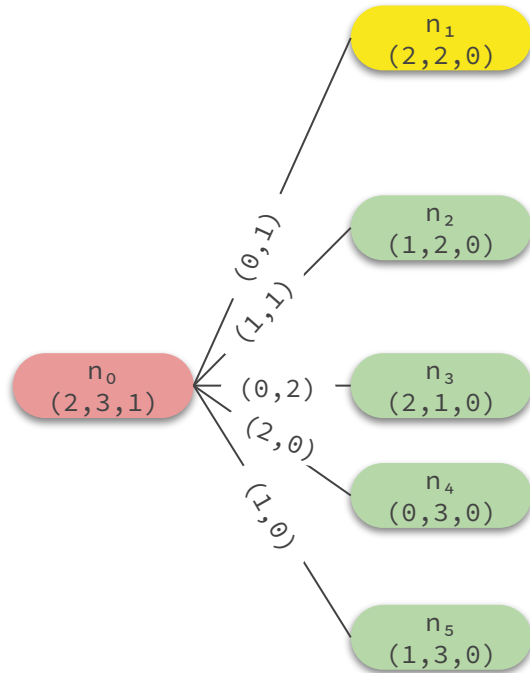
— — —



FRONTERA = $\{n_1, n_2, n_3, n_4, n_5\}$



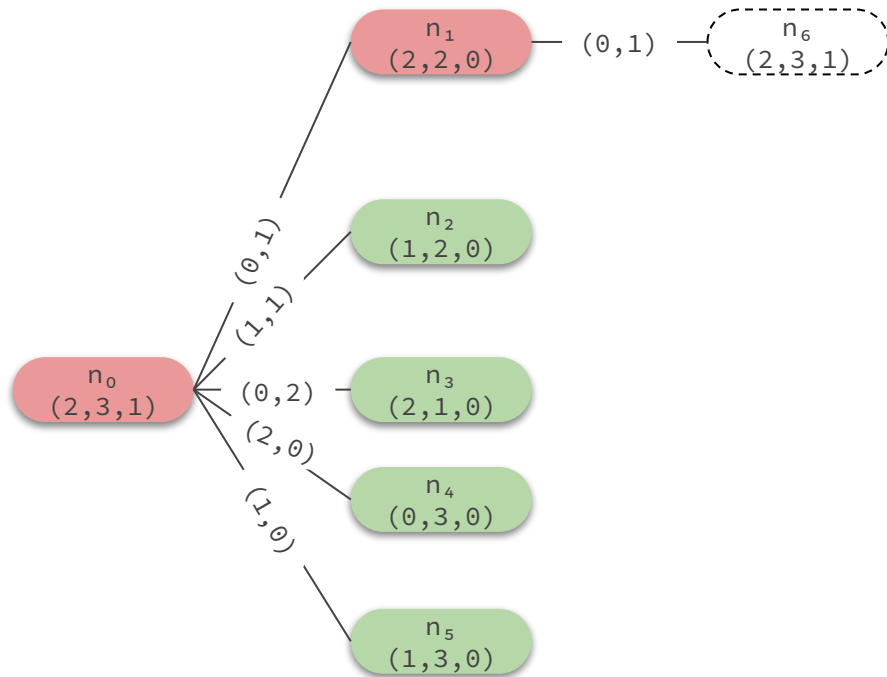
Ejemplo – Cruce de río



FRONTERA = $\{n_2, n_3, n_4, n_5\}$



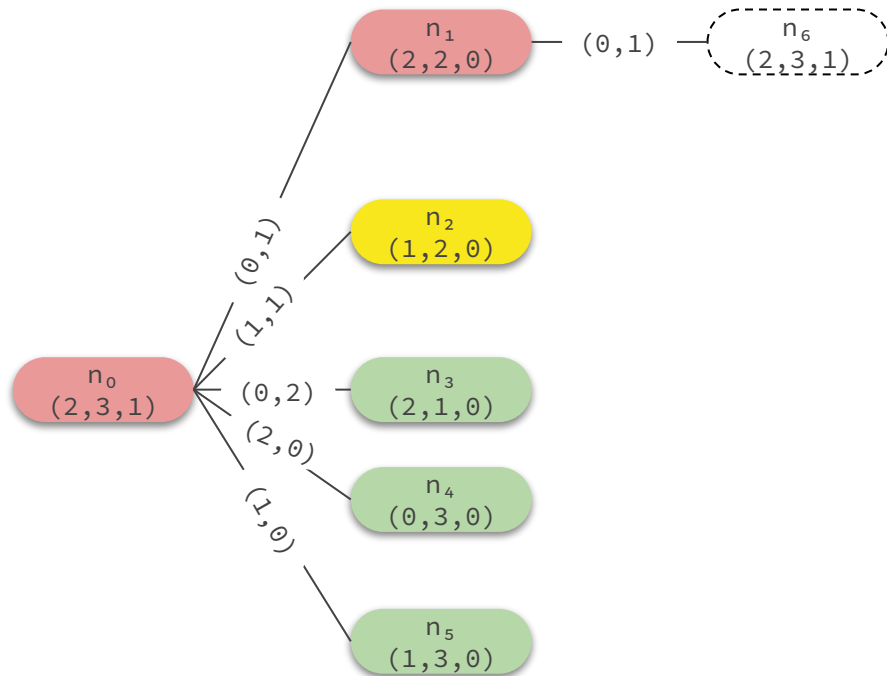
Ejemplo – Cruce de río



FRONTERA = $\{n_2, n_3, n_4, n_5\}$



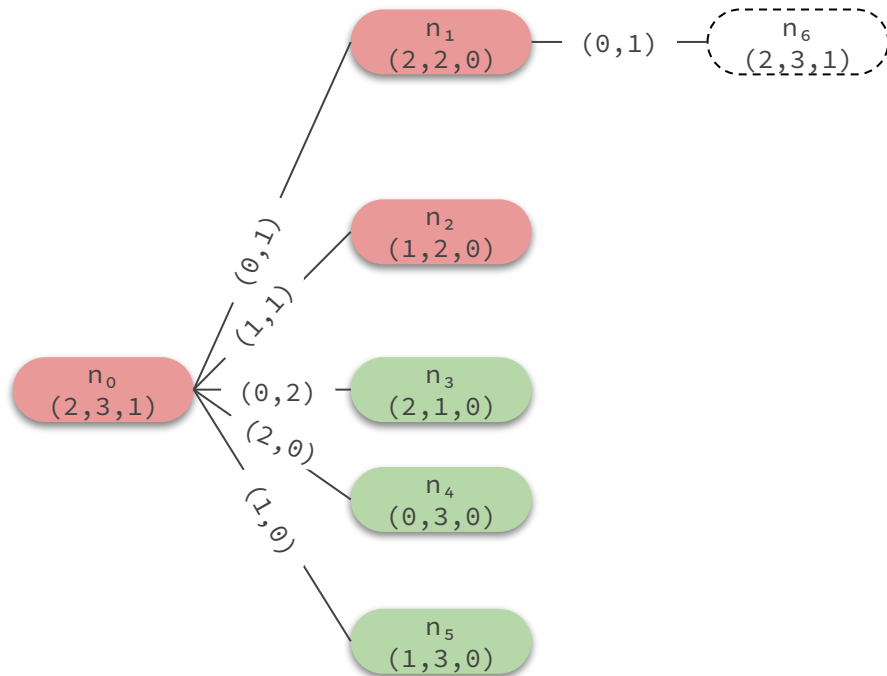
Ejemplo – Cruce de río



FRONTERA = $\{n_3, n_4, n_5\}$



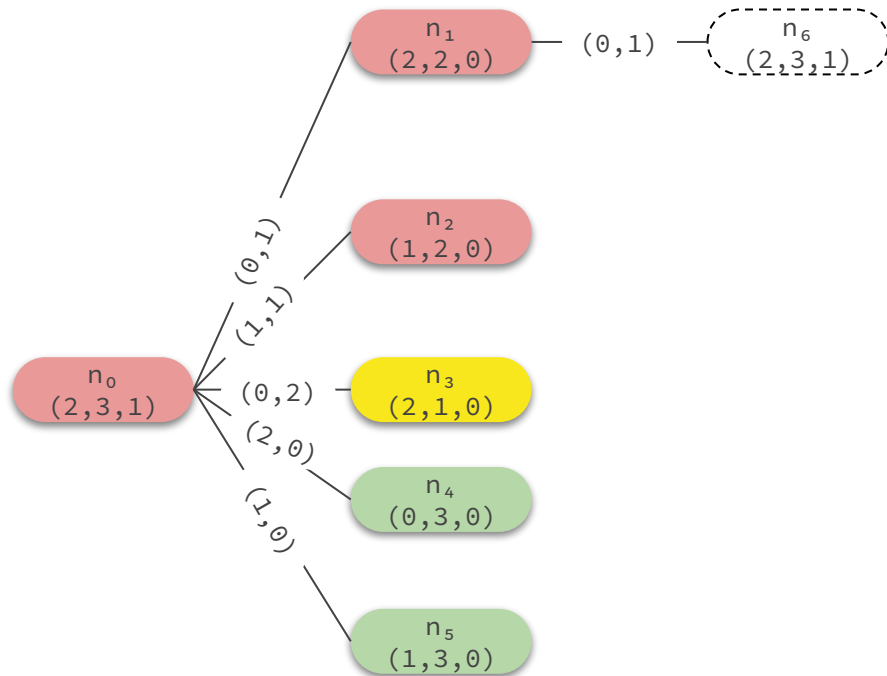
Ejemplo – Cruce de río



FRONTERA = $\{n_3, n_4, n_5\}$

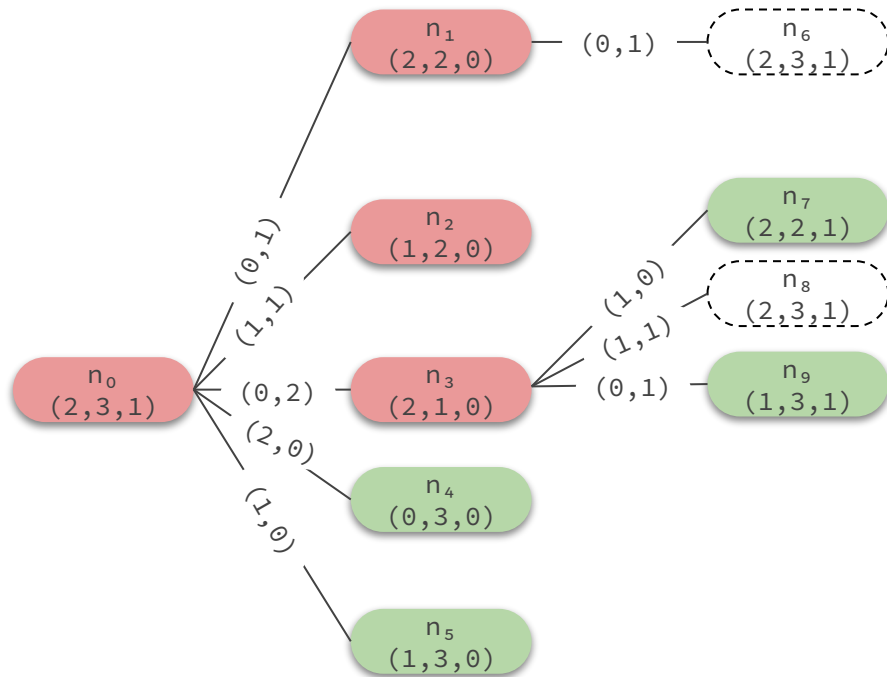


Ejemplo – Cruce de río





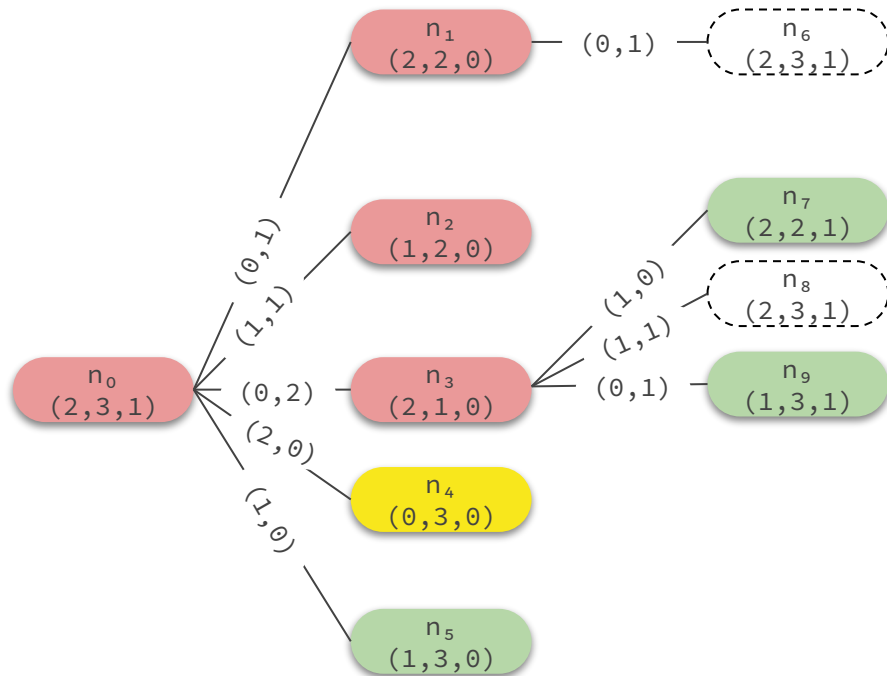
Ejemplo – Cruce de río



FRONTERA = $\{n_4, n_5, n_7, n_9\}$



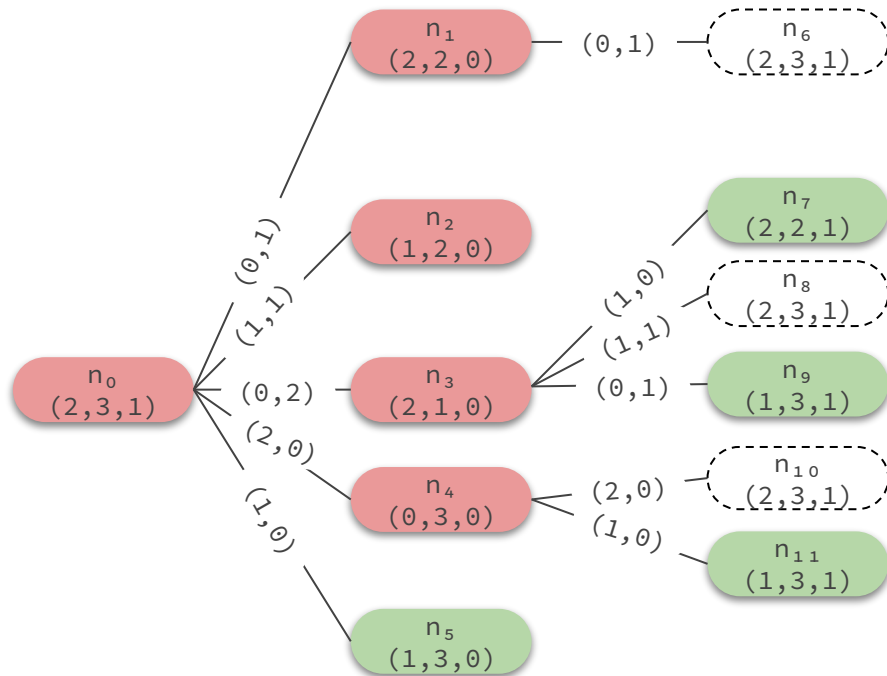
Ejemplo – Cruce de río



FRONTERA = $\{n_5, n_7, n_9\}$



Ejemplo – Cruce de río



FRONTERA = $\{n_5, n_7, n_9, n_{11}\}$

Los nodos n_9 y n_{11} repiten el estado (1,3,1), sin generar caminos cíclicos 🤖.

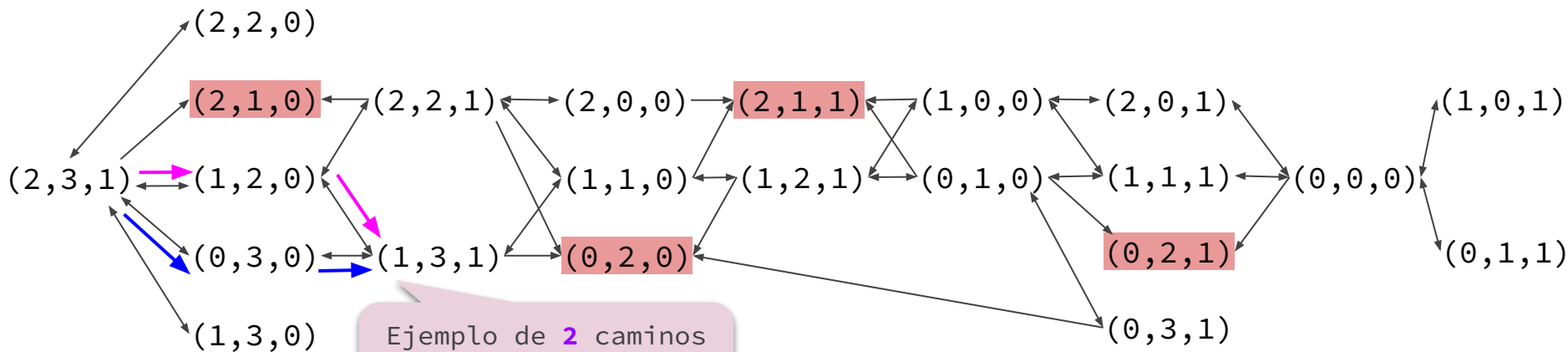
¿Esto qué significa? 🤔

Caminos redundantes

- En el grafo de estados, dos caminos **distintos** con mismo origen y mismo destino se llaman **caminos redundantes**.
- La existencia de caminos redundantes puede generar que el árbol de búsqueda construido por **TREE-SEARCH** tenga nodos con estados repetidos, incluso si se detectan ciclos.
- Un estado podría repetirse en el árbol de búsqueda tantas veces como caminos redundantes haya desde el estado inicial hasta a él.



Ejemplo – Cruce de río



Ejemplo de 2 caminos
redundantes, ambos
de largo 2.

En consecuencia, el estado
 $(1,3,1)$ se repetirá 2
veces en el nivel 2 del
árbol de búsqueda.

¿Qué tan grave
puede ser
esto? 🤔

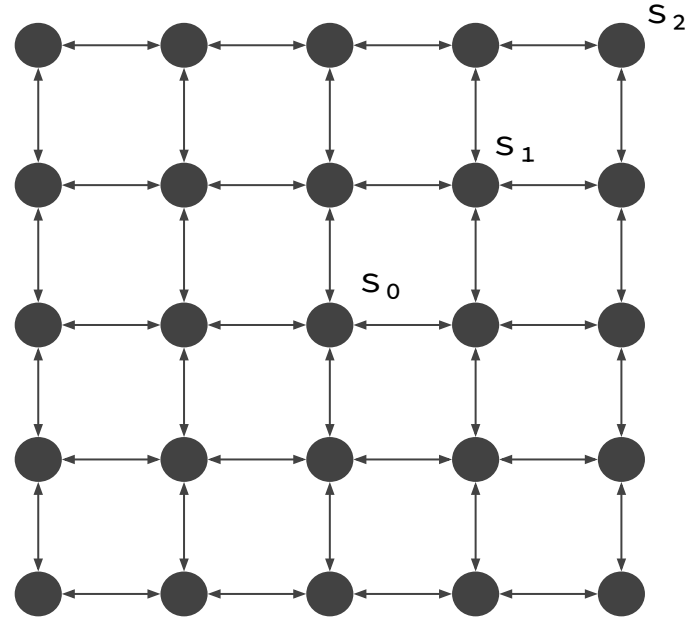


Ejemplo – Caminos redundantes

— — —

Sea un problema con un grafo de espacio de estados en forma de grilla.

¿Cuántos caminos redundantes de largo 2 hay entre s_0 y s_1 ?



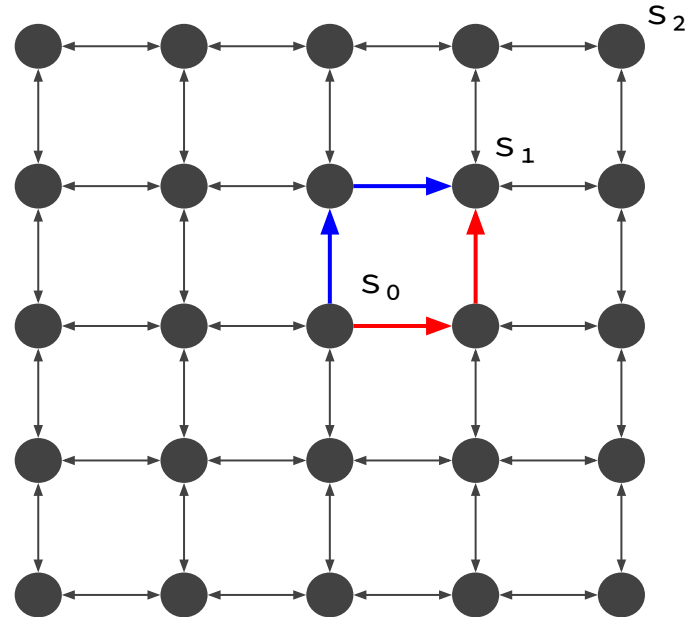


Ejemplo – Caminos redundantes

Sea un problema con un grafo de espacio de estados en forma de grilla.

¿Cuántos caminos redundantes de largo 2 hay entre s_0 y s_1 ?

Hay 2.





Ejemplo – Caminos redundantes

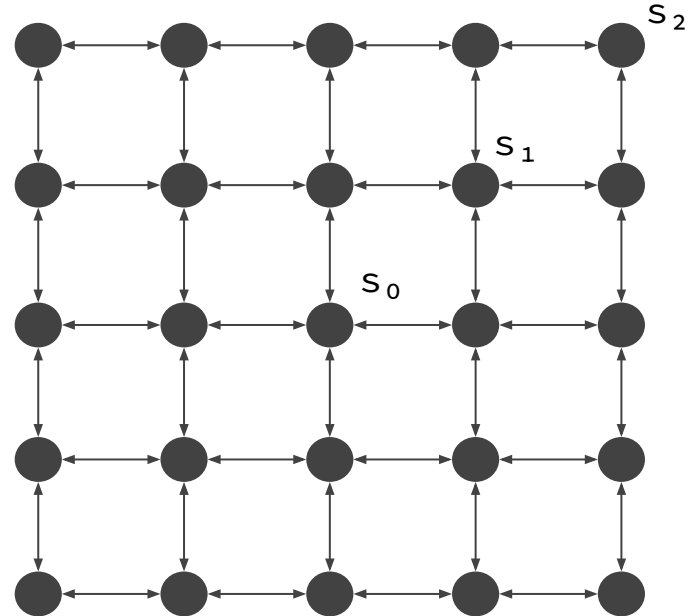
— — —

Sea un problema con un grafo de espacio de estados en forma de grilla.

¿Cuántos caminos redundantes de largo 2 hay entre s_0 y s_1 ?

Hay 2.

¿Cuántos caminos redundantes de largo 4 hay entre s_0 y s_2 ?





Ejemplo – Caminos redundantes

Sea un problema con un grafo de espacio de estados en forma de grilla.

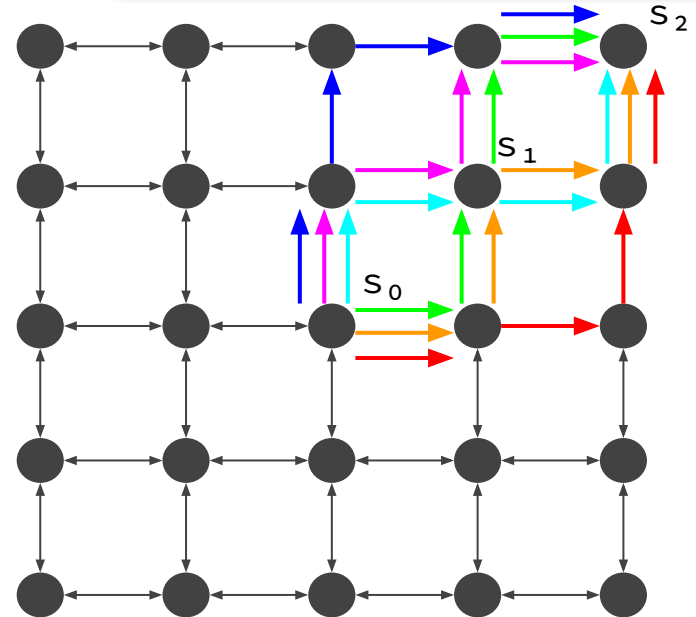
¿Cuántos caminos redundantes de largo 2 hay entre s_0 y s_1 ?

Hay 2.

¿Cuántos caminos redundantes de largo 4 hay entre s_0 y s_2 ?

Hay 6.

Es decir, en el nivel 2 del árbol de búsqueda construido por **TREE-SEARCH**, habrá 2 nodos con el estado s_1 . En el nivel 4, habrá 6 nodos con el estado s_2 ... **y esto empeora exponencialmente con el nivel.** 🤖





Ejemplo – Caminos redundantes

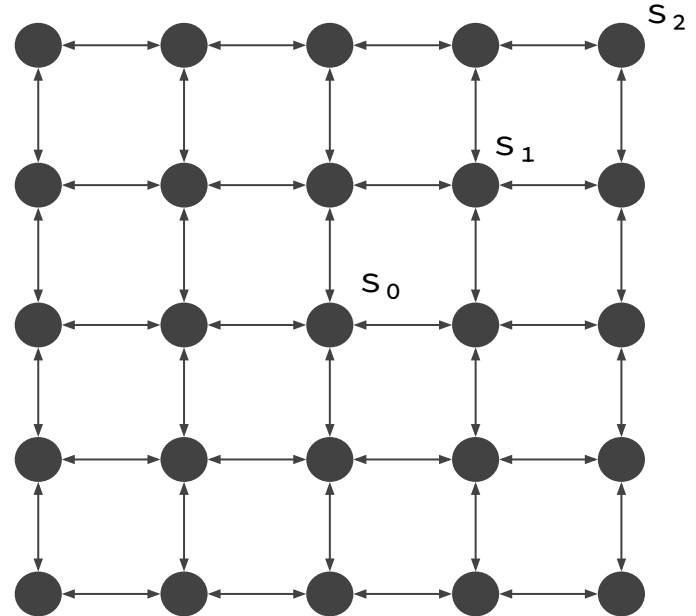
— — —

El número de estados a distancia a lo sumo d de s_0 responde a la fórmula:

$$2d^2 + 2d + 1$$

Debido a los caminos redundantes, el número de nodos del árbol de búsqueda hasta el nivel d (sin detectar ciclos) responde a la fórmula:

$$1 + 4^1 + \dots + 4^d = (4^{d+1} - 1)/(4 - 1)$$





Explosión combinatoria

— — —

d	Estados $2d^2 + 2d + 1$	Nodos $(4^{d+1} - 1) / (4 - 1)$	Tiempo
10	221	1.398.101	1,4 seg
15	481	$1,43 \times 10^9$	24 min
20	841	$1,46 \times 10^{12}$	17 días
25	1301	$1,5 \times 10^{15}$	48 años

Se asumen 1M de nodos generados por segundo.



Explosión combinatoria

— — —

d	Estados $2d^2+2d+1$	Nodos $(4^{d+1} - 1) / (4 - 1)$	Tiempo
10	221	1.398.101	1,4 seg
15	481	$1,43 \times 10^9$	24 min
20	841	$1,46 \times 10^{12}$	17 días
25	1301	$1,5 \times 10^{15}$	48 años

Se asumen 1M de nodos generados por segundo.

Suponiendo una memoria suficiente para almacenar los nodos necesarios, **TREE-SEARCH** difícilmente pueda encontrar en un tiempo razonable una solución óptima a distancia 20 en un espacio de estados en forma de grilla.

¿Es posible detectar caminos redundantes? 🤔



Detección de caminos redundantes

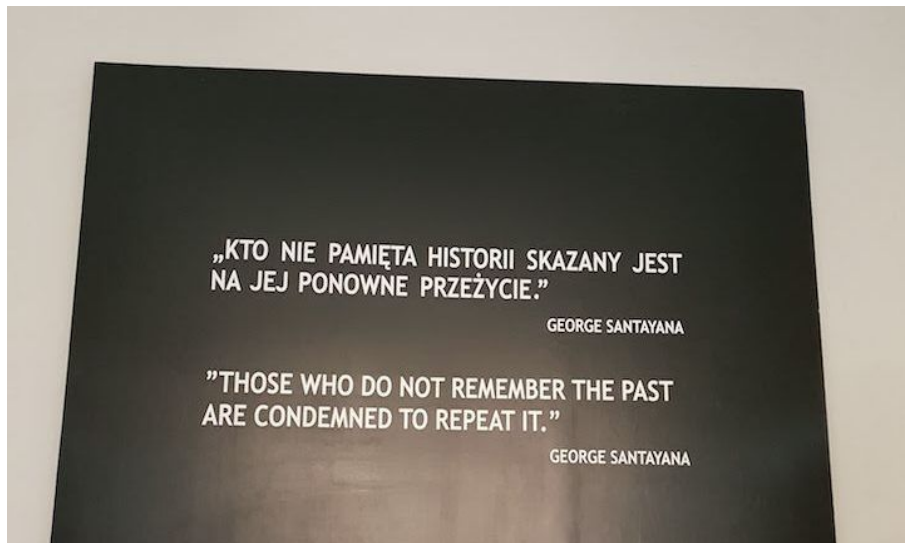
— — —

Para evitar cualquier camino redundante hay que recordar los estados **alcanzados** en el pasado, es decir, volver a la búsqueda de grafo.

“Los algoritmos que olvidan su historia están condenados a repetirla” – Stuart Russell y Peter Norvig en *Inteligencia Artificial: un enfoque moderno*.

Un poco de historia...

— — —



“Aquellos que no pueden recordar el pasado están condenados a repetirlo” – George Santayana en *La vida de la razón* (1905).

Placa a la entrada del Pabellón 4 de Auschwitz.



GRAPH-SEARCH vs TREE-SEARCH

— — —

Decidir cuál algoritmo es más indicado para un problema no siempre es sencillo.

- Si el espacio de estados tiene pocos caminos redundantes, o ninguno, entonces **TREE-SEARCH** será más eficiente.
- Por el contrario, si el espacio de estados tiene muchos caminos redundantes, la única alternativa será **GRAPH-SEARCH**.

Criterios de evaluación para algoritmos de búsqueda

1. **Compleitud.**

Si hay solución,
¿encuentra al menos una?

2. **Optimalidad.**

Si hay solución,
¿encuentra la óptima?

3. **Tiempo.**

¿Cuántos nodos genera?

4. **Memoria.**

¿Cuánta memoria usa?

— — —



Próximamente

Para dar una implementación de **GRAPH-SEARCH** y **TREE-SEARCH** tenemos que elegir:

1. Una ED para la frontera.
2. El criterio en que se eligen los nodos de la frontera.

La decisión tendrá impacto en la performance del algoritmo.